WHO WROTE IT?

Identifying Large Language Models from Their Responses

CMPSC 448 - Machine Learning and Artificial Intelligence

MIDTERM PROJECT REPORT

The Pennsylvania State University

University Park, Pennsylvania

Fall 2026

Prepared By

Palash Doshi *(Team Leader)*

Siddharth Pendyala

Submission Date: October 11, 2026

GitHub Repository:

https://github.com/palashDoshi1/CMPSC-448-Midterm-Project

Abstract

The rapid development and advancement of novel large language models (LLMs) has made AI-generated text increasingly common in education, software development, finance, and everyday communication. Although models such as GPT, Claude, and Gemini often generate similar responses, they can be seen to exhibit certain distinct linguistic, structural, and stylistic characteristics. This project investigates whether neural-network classifiers can identify which LLM generated a response based on its text.

We constructed an original, balanced dataset containing 540 responses from GPT, Claude, and Gemini across six categories: computer science and software, finance and banking, mathematics and logic, sports, stocks and trading, and general writing. We implemented Convolutional Neural Network (CNN) and Long Short-Term Memory (LSTM) classifiers to investigate LLM authorship identification, the influence of user prompts, cross-domain generalization, and model-specific writing characteristics.

On the primary output-only classification task, the CNN achieved 82.72% test accuracy and 0.8293 macro-F1, compared with 71.60% accuracy and 0.7141 macro-F1 for the LSTM. Both models achieved 33.33% accuracy using prompts alone, as deterministic predictions yield one correct label for each prompt paired with all three classes. Combined input performed worse under the tested token allocation, but it also reduced the maximum response length from 512 to 383 tokens. The reported performance differences are descriptive and were not established as statistically significant.

For cross-domain generalization, both classifiers were trained without general-writing responses and evaluated on 90 held-out writing examples. The CNN achieved 72.22% accuracy and 0.7161 macro-F1, compared with 56.67% accuracy and 0.5429 macro-F1 for the LSTM.

Overall, our findings suggest that LLM-generated responses contain identifiable patterns, although classification performance varies across architectures and task domains.

1. Introduction and Motivation

1.1 Background and Research Problem

Large language models (LLMs) have become an important part of modern technology, supporting applications in education, software development, financial analysis, research, and everyday communication. Models such as GPT, Claude, and Gemini can generate detailed explanations, solve mathematical problems, assist with programming, and produce natural-sounding written content. As these systems become increasingly capable, and due to this, identifying the source of AI-generated responses presents an interesting machine-learning challenge.

Although different LLMs can provide similar information when responding to the same question, their responses may differ in vocabulary, sentence structure, formatting, explanation style, and level of detail. For example, one model may prefer structured explanations with headings and bullet points, while another may provide a more concise or conversational response. These differences could represent recognizable linguistic and structural patterns, sometimes described as model-specific fingerprints.

Our project investigates whether neural-network classifiers can learn these patterns and identify which LLM generated a particular response. We approach this problem as a supervised multiclass text-classification task, where each response belongs to one of three classes: GPT, Claude, or Gemini.

1.2 Our Motivation and Initial Research Discussions

Our interest in this project developed through discussions about artificial intelligence, machine learning, and how different language models generate responses. As computer science students, we regularly encounter AI-generated explanations, programming assistance, and answers to technical questions. This made us curious about whether different LLMs have identifiable writing characteristics, even when they respond to the same prompt.

During our initial planning discussions, we considered how to approach the problem beyond simply comparing the quality or correctness of AI-generated answers. We were more interested

in understanding whether a machine-learning classifier could recognize patterns that humans might not immediately notice. In particular, we wanted to investigate whether GPT, Claude, and Gemini could be distinguished based on how they structure and communicate information.

Another important consideration was choosing a dataset that would reflect a variety of topics rather than a single subject. We selected six categories connected to our interests: computer science and software, finance and banking, mathematics and logic, sports, stocks and trading, and general writing. These categories provided a combination of technical, quantitative, factual, and open-ended questions.

We also decided to construct our dataset in-house instead of relying on an existing classification dataset. This allowed us to design the prompts, collect responses from all three LLM families, and maintain a balanced number of examples for each model. By collecting responses to the same prompts, we could investigate the differences in writing characteristics while reducing the influence of differences in the questions themselves.

Beyond the classification task, we were also motivated by the opportunity to experience the complete machine-learning workflow. Instead of focusing only on just model training, we wanted to understand how components such as dataset construction, preprocessing, architecture selection, evaluation, and experimental design contribute to the reliability of machine-learning results.

1.3 Research Objectives and Questions

The primary objective of this project is to determine whether neural-network classifiers can accurately identify which large language model generated a response. To investigate this problem, we implemented and evaluated two architectures: a Convolutional Neural Network (CNN) and a Long Short-Term Memory (LSTM) network.

We organized our investigation around the following four research questions.

RQ1 - Can we identify which LLM generated a response?

We investigate whether CNN and LSTM classifiers can distinguish between GPT, Claude, and Gemini using only their generated responses. We compare classification accuracy, macro-F1 scores, and confusion matrices to evaluate the performance of both architectures.

RQ2 - Does the user's prompt help identify the LLM?

We examine whether the original user prompt provides useful information for identifying the generating model. We compare three input configurations: input-only, output-only, and combined input/output. This allows us to investigate whether including the prompt improves classification performance.

RQ3 - Do LLM fingerprints generalize across tasks or domains?

We investigate whether classifiers (both CNN and LSTM) trained on responses from five categories can identify LLMs in a previously unseen sixth category. Specifically, we exclude general-writing responses from training the models in this session and evaluate both architectures on the held-out general-writing domain.

RQ4 - What characteristics distinguish different LLMs?

We analyzed linguistic and structural characteristics that may distinguish GPT, Claude, and Gemini, including response length, vocabulary usage, lexical diversity, and formatting patterns. This analysis helps us better understand which characteristics may contribute to successful LLM identification.

Together, these research questions allow us to examine not only whether LLM authorship classification is possible, but also which information is most useful, how well learned patterns generalize across domains, and what writing characteristics distinguish different LLMs.

2. Project Development and Timeline

2.1 October 8, 2026 - Initial Discussions and Research Planning

Our project began with discussions about identifying the source of text generated by different large language models. We were interested in exploring whether GPT, Claude, and Gemini produced distinct responses for a machine-learning classifier to recognize the generating model. Rather than evaluating which LLM was more accurate or useful, we focused on whether differences in writing style, structure, and vocabulary could serve as identifiable patterns.

During our initial planning, we considered the scope of the problem, the type of dataset required, and the neural-network architectures we could implement. We decided to approach the task as a three-class text-classification problem and compare two architectures: a Convolutional Neural Network (CNN) and a Long Short-Term Memory (LSTM) network.

We also discussed how to make the project more meaningful than a basic classification exercise. In addition to identifying the generating model, we wanted to investigate whether the original prompt influenced classification, whether learned patterns generalized across different subjects, and which writing characteristics distinguished the three LLMs. These discussions helped establish the four research questions that guided our experiments.

2.2 October 9, 2026 - Original Dataset Construction

After defining our research objectives, we focused on constructing our own dataset rather than relying on a pre-existing collection of LLM-generated responses. We wanted control over the prompts, the subject categories, and the balance between the three models.

We selected six categories based on our academic interests and the types of questions people commonly ask language models: computer science and software, finance and banking, mathematics and logic, sports, stocks and trading, and general writing. These categories allowed us to include technical explanations, quantitative reasoning, factual questions, and open-ended writing tasks.

We developed 180 original prompts (30 for each topic) and collected a response from each of the three LLM families for every prompt. This resulted in a balanced dataset of 540 responses, containing 180 examples each from GPT, Claude, and Gemini. Within these model families, we specifically used GPT-6 Instant, Claude Sonnet 5.5, and Gemini 1.5 Pro.

Each dataset record included a prompt identifier, category, generating model, original input prompt, and generated output. Maintaining the same prompts across the three models was important because it allowed us to investigate differences in their responses without introducing different questions for different classes.

The dataset was also organized into a consistent structure and prepared for model training and evaluation. We also considered the importance of avoiding data leakage, particularly because each original prompt was associated with three responses. To address this, we used prompt-grouped data splits so that responses associated with the same prompt would not appear across different training, validation, and test sets.

2.3 October 10, 2026 - Model Implementation and Initial Experiments

Once the dataset was prepared, we began implementing the two neural-network architectures. We divided the main modeling responsibilities, with Palash focusing on the CNN implementation and Siddharth focusing on the LSTM implementation.

The CNN was designed to learn local textual patterns through convolutional filters, while the LSTM was designed to process token sequences and learn patterns across the response text. Both architectures were developed for the same three-class classification task, allowing us to compare their performance on responses generated by GPT, Claude, and Gemini.

Our implementation process included tokenization, vocabulary construction, numerical encoding, sequence padding and truncation, embedding layers, model training, validation, and evaluation. We used PyTorch to implement the neural networks and Google Colab to execute our experiments.

We initially focused on the primary research question: whether generated responses alone contained enough information to identify the source LLM. We trained the models, monitored validation performance, and evaluated their ability to classify previously unseen responses.

This stage also involved debugging, checking dataset compatibility, reviewing training behavior, and organizing the code so that the experiments could be reproduced and compared.

2.4 October 11, 2026 - Additional Experiments, Evaluation, and Finalization

Following the initial classification experiments, we extended our investigation to the remaining research questions.

For RQ2, we examined three input configurations: the original prompt alone, the generated response alone, and the combination of the prompt and response. This allowed us to investigate whether including the user's original question improved the ability to identify the generating LLM.

For RQ3, we investigated cross-domain generalization by excluding general-writing responses from model training and validation. We then evaluated the trained CNN and LSTM classifiers on 90 general-writing responses that were not included in training. This experiment helped us examine whether patterns learned from other categories could transfer to a previously unseen subject domain.

For RQ4, we developed a separate Jupyter notebook dedicated to analyzing the linguistic and structural characteristics of responses generated by GPT, Claude, and Gemini. Both team members collaborated on this analysis, examining response length, lexical diversity, punctuation, and formatting patterns. Separating RQ4 from the CNN implementation helped us organize the descriptive analysis independently from the neural-network classification experiments.

We also reviewed model accuracy, macro-F1 scores, confusion matrices, and training behavior to interpret the experimental results. Particular attention was given to understanding why the CNN and LSTM performed differently and identifying limitations in the dataset and evaluation process.

As the project approached completion, we organized our implementations and supporting files in a shared GitHub repository. We also began preparing the final report to document our methodology, results, interpretation, and lessons learned.

2.5 Team Collaboration and Division of Responsibilities

Collaboration was an important part of our project development process. Both team members contributed to defining the research problem, discussing the experimental approach, and developing the overall project structure.

We divided the primary model implementation responsibilities to make the development process more efficient. Palash focused on implementing and evaluating the CNN, while Siddharth focused on implementing and evaluating the LSTM. When it came to RQ4, both of us worked on it together.

This division allowed us to investigate two different neural-network approaches in parallel while maintaining a common research objective. We coordinated our experiments through the shared dataset, discussed the results, and worked toward a consistent evaluation framework.

Beyond implementing the models, our work involved constructing and organizing the dataset, reviewing preprocessing decisions, debugging training issues, interpreting experimental results, maintaining the GitHub repository, and preparing the final report.

This collaborative approach allowed us to combine our individual work into a single investigation of LLM authorship classification.

2.6 Challenges Encountered and Lessons from Development

One of the main challenges we encountered was constructing a dataset that was sufficiently balanced and consistent for comparing the three LLM families. Because we developed the dataset ourselves, we needed to ensure that each prompt was associated with responses from all three models and that the resulting data could be processed consistently.

Another important challenge involved preventing data leakage. Since multiple responses were from the same prompt, randomly splitting individual responses could allow the same question to appear in both training and testing. Using prompt-grouped splits helped reduce this risk.

Model training and evaluation also required careful decisions about preprocessing, sequence length, vocabulary construction, hyperparameters, and checkpoint selection. We learned that high training performance does not necessarily translate into strong performance on unseen examples.

The cross-domain experiments further demonstrated the importance of evaluating models beyond their original training distribution. A classifier that performs well on familiar categories may not achieve the same accuracy when applied to responses from a different subject area.

Overall, the development process strengthened our understanding of the complete machine-learning workflow, from formulating a research question and constructing a dataset to implementing models, conducting experiments, interpreting results, and documenting our findings.

3. Dataset Construction and Preprocessing

3.1 Dataset Design and Collection

A major component of our project was developing an original dataset specifically designed for identifying the source of LLM-generated responses. Rather than relying on a publicly available dataset, we created our own collection of prompts and obtained responses from three large language model families: GPT, Claude, and Gemini. Within these model families, we specifically used GPT-6 Instant, Claude Sonnet 5.5, and Gemini 1.5 Pro.

Responses were collected manually through normal conversations with the three models using their chat interfaces. Each project prompt was submitted to GPT, Claude, and Gemini, and the generated responses were copied and pasted into the dataset. Each response was stored alongside its original prompt, model label, category, and prompt identifier. The formatting measurements reported in RQ4 describe the text as stored after copying and pasting.

The submitted artifacts do not record a standardized conversation-reset procedure, generation settings, or detailed copying and cleaning logs. These collection conditions therefore cannot be fully reconstructed from the dataset and notebooks.

We developed 180 prompts across six categories, with 30 prompts in each category. Each prompt was submitted to all three LLMs, producing three corresponding responses. This resulted in a balanced dataset containing 540 responses, with 180 examples from each model.

Using identical prompts across all three models was an important design decision. It allowed us to compare responses generated under similar task conditions and reduced the possibility that the classifier would simply learn differences between the questions assigned to different models.

The dataset included both technical and nontechnical topics to encourage variation in response content, structure, and writing style. We aimed to include tasks involving explanations, reasoning, factual knowledge, and open-ended writing.

3.2 Dataset Categories and Distribution

We selected six categories based on our academic interests and the types of tasks for which LLMs are commonly used.

Table 1. Dataset distribution by category

Category	Prompts	GPT	Claude	Gemini	Total responses
Computer Science and Software	30	30	30	30	90
Finance and Banking	30	30	30	30	90
Mathematics and Logic	30	30	30	30	90
Sports	30	30	30	30	90
Stocks and Trading	30	30	30	30	90
General Writing	30	30	30	30	90
Total	**180**	**180**	**180**	**180**	**540**

The dataset was balanced across both model classes and subject categories. Each category contributed 90 responses, and each model contributed one-third of the total dataset.

This balance was important because it prevented the classifier from obtaining high accuracy simply by predicting a majority class.

3.3 Dataset Structure and Features

Each dataset record contained five fields:

Table 2. Dataset attributes

Attribute	Description
`prompt_id`	Unique identifier assigned to each original prompt
`category`	Subject category associated with the prompt
`model`	Generating LLM label: GPT, Claude, or Gemini
`input`	Original prompt submitted to the LLM
`output`	Text response generated by the LLM

The `model` field served as the target label for supervised classification.

The `input` and `output` fields were used to investigate different configurations in RQ2. Specifically, we evaluated classification using the original prompt alone, the generated response alone, and a combination of the two.

This structure also supported RQ3, where the `category` field was used to separate training domains from the held-out general-writing domain.

3.4 Training, Validation, and Test Splits

To evaluate model performance on unseen examples, we divided the dataset into training, validation, and test sets.

For the primary classification experiments, we used the following distribution:

Table 3. Primary dataset split

Dataset split	Responses	Percentage
Training	378	70.00%
Validation	81	15.00%
Test	81	15.00%
Total	**540**	**100%**

A key consideration was preventing data leakage between these splits.

Because each prompt generated three responses, a conventional random split at the individual-response level could place responses associated with the same original prompt in different subsets.

Both notebooks used the same predefined train.csv, validation.csv, and test.csv files. All three responses associated with a prompt_id belonged to the same partition, and prompt IDs were disjoint across partitions. The training, validation, and test sets contained 126, 27, and 27 unique prompts, respectively, corresponding to 378, 81, and 81 responses. The prompt_id values stored in the shared CSV files preserve the exact assignments used by both architectures.

The notebooks load these predefined partitions rather than generating new splits. The original split-generation code and random seed are not included in the submitted artifacts; the shared CSV files and their prompt identifiers are therefore the reproducibility record for the partition assignments. The training seed reported in Section 4.7 should not be interpreted as the original split-generation seed.

This ensured that the model was evaluated on responses to prompts that were not present in its training data.

The training set was used to learn model parameters, the validation set was used for model selection and early stopping, and the test set was reserved for final performance evaluation.

3.5 Text Tokenization and Vocabulary Construction

Before training the neural networks, the raw text responses needed to be converted into numerical representations.

The classification tokenizer used the case-sensitive regular expression r"\w+|[^\w\s]", producing word tokens and individual punctuation or symbol tokens. Whitespace, including line breaks, was not retained as explicit tokens. This tokenizer differs from the word-counting procedure used for the descriptive RQ4 analysis.

For the primary experiments, we constructed a vocabulary containing up to 10,000 tokens. The vocabulary included two reserved tokens:

- `PAD` (index 0), used to pad shorter sequences.
- `UNK` (index 1), used for tokens not included in the vocabulary.

Each text sequence was converted into a list of integer token IDs using this vocabulary.

To provide consistent input dimensions, sequences were padded or truncated to a maximum length of 512 tokens.

The resulting numerical sequences were passed to the embedding layers of the CNN and LSTM models, which transformed token IDs into dense vector representations.

Vocabulary construction used only the training partition for each experiment. The RQ1 output-only vocabulary was constructed from training responses. For RQ2, separate vocabularies were constructed from training prompts and from combined training prompt–response sequences using the specified token allocation. For RQ3, vocabulary construction used only source-domain training responses, excluding general writing. Validation and test text did not contribute to vocabulary selection or token-frequency calculations. Both

architectures used matching vocabulary mappings for corresponding experiments, and tokens absent from a vocabulary were mapped to UNK.

3.6 Data Preparation for the Research Questions

The dataset was prepared differently depending on the research question being investigated.

For RQ1, the models received only the generated responses (`output`). This configuration tested whether the writing characteristics of the response alone were sufficient for identifying the generating LLM.

For RQ2, we evaluated three configurations: input-only, output-only, and combined input/output. The input-only configuration used the original prompt, while the combined configuration included both the prompt and generated response.

For combined input/output, we retained up to 128 prompt tokens, a dedicated [SEP] token, and up to 383 response tokens, within a maximum sequence length of 512. Unused prompt capacity was not reassigned to the response. Thus, the maximum retained response length decreased from 512 tokens in output-only classification to 383 tokens in combined classification. The comparison changes both prompt inclusion and the amount of response information available.

For RQ3, we removed general-writing rows from each predefined partition while preserving the other prompt assignments, leaving 315 source-domain training responses, 69 validation responses, and 66 test responses. Fresh CNN and LSTM models and a training-only source-domain vocabulary were used. All 90 general-writing responses across the original partitions formed the unseen-domain test set; none were used for training, vocabulary construction, or checkpoint selection.

For RQ4, we examined linguistic and structural characteristics of the generated text to identify possible differences among the three LLM families.

These configurations allowed us to investigate multiple aspects of LLM authorship classification while using a consistent underlying dataset.

3.7 Data Quality and Leakage Prevention

We took several steps to improve the reliability of our experiments.

First, we maintained a balanced dataset containing the same number of examples from GPT, Claude, and Gemini.

Second, we used identical prompts across the three models to reduce differences caused by the questions themselves.

Third, we applied prompt-grouped splitting so that responses associated with the same prompt did not appear in both training and test sets.

Fourth, we separated the general-writing domain from training and validation in the cross-domain experiments. This allowed us to evaluate model performance on a subject category not encountered during training.

Finally, every experiment constructed its vocabulary from its own training partition only. Validation and test text were excluded from vocabulary selection and token-frequency calculations.

These design decisions helped create a more reliable experimental framework for comparing the CNN and LSTM architectures.

4. Neural Network Architectures and Training Methodology

4.1 Overview of the Two Architectures

To investigate whether neural networks can identify the source of LLM-generated responses, we implemented two text-classification architectures: a Convolutional Neural Network (CNN) and a Long Short-Term Memory (LSTM) network.

Both models were designed to classify text into three categories: GPT, Claude, and Gemini. They received tokenized text sequences and produced three classification scores corresponding to the possible generating models.

We selected these architectures because they learn different types of textual patterns. The CNN uses convolutional filters to identify local patterns in sequences of word embeddings, while the LSTM processes tokens sequentially and maintains a hidden state that can capture information across the text.

Comparing these architectures allowed us to investigate whether local textual patterns or sequential representations were more effective for LLM authorship identification.

4.2 Convolutional Neural Network (CNN)

The CNN architecture was implemented and evaluated by Palash Doshi using PyTorch.

The model was designed as a TextCNN classifier, using an embedding layer followed by parallel one-dimensional convolutional layers, pooling, dropout, and a fully connected classification layer. [1]

4.2.1 Embedding Layer

Each input sequence was represented as a series of integer token IDs. These token IDs were passed through an embedding layer that transformed them into dense vectors of 128 dimensions.

The embedding layer allowed the model to learn numerical representations of words and punctuation tokens during training.

4.2.2 Convolutional Layers

The CNN used three parallel convolutional branches with kernel sizes of 3, 5, and 7.

Each branch contained 128 convolutional filters.

These different kernel sizes allowed the model to examine local token patterns of varying lengths. For example, smaller kernels could capture short expressions, while larger kernels could capture longer local combinations of words.

4.2.3 Pooling and Feature Extraction

The CNN applied the rectified linear unit (ReLU) activation after each convolution. Before global max pooling, activations at padded token positions were masked so that they could not determine the pooled features. Max pooling then selected the strongest activation from each filter across the valid token positions.

The pooled features from the three convolutional branches were concatenated into a single feature vector.

Because each branch produced 128 features, the combined representation contained 384 features.

4.2.4 Dropout and Classification

A dropout rate of 0.3 was applied before the final classification layer to reduce overfitting.

The fully connected layer mapped the extracted features to three output scores representing GPT, Claude, and Gemini.

The predicted class was determined by selecting the class with the highest output score.

4.2.5 CNN Architecture Summary

Input text → Tokenization → Embedding (128 dimensions) → Parallel Convolutions (3, 5, 7) → ReLU → Padding Mask → Global Max Pooling → Feature Concatenation (384 dimensions) → Dropout (0.3) → Fully Connected Layer → GPT / Claude / Gemini

4.3 Long Short-Term Memory Network (LSTM)

The LSTM architecture was implemented and evaluated by Siddharth Pendyala using PyTorch.

Unlike the CNN, which extracts local textual features using convolutional filters, the LSTM processes input tokens sequentially and updates an internal hidden state. [2]

This architecture was selected to investigate whether sequential information could help identify differences in the writing characteristics of GPT, Claude, and Gemini.

4.3.1 Embedding Layer

The LSTM used an embedding dimension of 128, converting token IDs into dense numerical representations.

4.3.2 LSTM Layer

The model contained a single unidirectional LSTM layer with 128 hidden units.

The LSTM processes the token sequence in order, updating its internal representations as it encounters additional tokens.

The implementation used packed padded sequences based on the actual sequence lengths, excluding right-padding positions from recurrent processing. The final hidden state of the single unidirectional LSTM layer served as the response representation passed to dropout and the classification layer.

4.3.3 Dropout and Classification

A dropout rate of 0.3 was applied after the LSTM representation.

The resulting representation was passed to a classification layer that produced three output scores corresponding to the three LLM classes.

The predicted class was selected using the highest output score.

4.3.4 LSTM Architecture Summary

Input text → Tokenization → Embedding (128 dimensions) → Packed Single-Layer Unidirectional LSTM (128 hidden units) → Final Hidden State → Dropout (0.3) → Fully Connected Layer → GPT / Claude / Gemini

4.4 Architecture Comparison

Although both architectures were designed for the same classification task, they differed in how they processed and represented text.

Table 4. Comparison of CNN and LSTM architectures

Feature	CNN	LSTM
Implemented by	Palash Doshi	Siddharth Pendyala
Architecture	TextCNN	Single-layer LSTM
Embedding dimension	128	128
Convolutional kernel sizes	3, 5, 7	Not applicable
Filters per kernel size	128	Not applicable
Hidden units	Not applicable	128
Directionality	Not applicable	Unidirectional
Dropout	0.3	0.3
Maximum sequence length	512 tokens	512 tokens
Output classes	3	3
Main feature-learning approach	Local textual patterns	Sequential representations

The CNN can identify patterns at different local sequence lengths through its parallel convolutional filters.

The LSTM processes the sequence in order and learns representations influenced by previously encountered tokens.

These differences provided a basis for comparing the effectiveness of convolutional and recurrent approaches to LLM authorship classification.

4.5 Model Training Configuration

Both architectures were trained using supervised learning.

Each training example consisted of an input text sequence and a corresponding class label identifying the generating LLM.

The models were optimized using the Adam optimizer and cross-entropy loss. [3]

For the primary experiments, the following training settings were used:

Table 5. Training hyperparameters

Hyperparameter	CNN	LSTM
Optimizer	Adam	Adam
Learning rate	0.001	0.001
Loss function	Cross-entropy	Cross-entropy
Batch size	16	16
Maximum epochs	20	20
Early stopping patience	4	4
Embedding dimension	128	128
Dropout rate	0.3	0.3
Vocabulary limit	10,000	10,000
Maximum sequence length	512	512

During training, the models processed batches of tokenized text, calculated classification loss, and updated their parameters using backpropagation.

After each epoch, performance was evaluated on the validation set.

Early stopping was used to reduce unnecessary training and limit overfitting when validation performance stopped improving.

For the primary output-only experiment, the CNN achieved its best validation loss at epoch 9, while the LSTM selected its best checkpoint at epoch 5 based on validation macro-F1.

For the LSTM cross-domain experiment, the best checkpoint was selected at epoch 12.

These checkpoints were used for the corresponding evaluations.

4.6 Evaluation Metrics

We evaluated classification performance using accuracy, macro-averaged F1 score, and confusion matrices.

4.6.1 Accuracy

Accuracy measures the proportion of correctly classified responses.

Accuracy = Number of correctly classified responses / Total number of responses

For example, correctly classifying 67 out of 81 responses gives an accuracy of approximately 82.72%.

4.6.2 Precision

Precision measures how many responses predicted as belonging to a particular model actually originated from that model.

Precision = True Positives / (True Positives + False Positives)

4.6.3 Recall

Recall measures how many responses belonging to a particular model were correctly identified.

Recall = True Positives / (True Positives + False Negatives)

4.6.4 F1 Score

The F1 score combines precision and recall into a single measure.

F1 = 2 × (Precision × Recall) / (Precision + Recall)

4.6.5 Macro-F1

Macro-F1 calculates the F1 score separately for GPT, Claude, and Gemini, then averages the three scores equally.

This metric was useful because it allowed us to evaluate performance across all three classes rather than focusing only on overall accuracy.

4.6.6 Confusion Matrices

Confusion matrices were used to examine which models were correctly identified and which pairs of LLMs were most frequently confused.

They helped us investigate class-specific errors and better understand differences between CNN and LSTM performance.

4.7 Model Selection and Reproducibility

The training experiments used random seed 42 for Python, NumPy and PyTorch, including CUDA seeding when available. Training was limited to 20 epochs, with early stopping after four consecutive epochs without improvement in the selected validation criterion. The selected checkpoint was restored before test evaluation.

For RQ1, the CNN selected the checkpoint with the lowest validation loss, while the LSTM selected the checkpoint with the highest validation macro-F1. The RQ2 output-only baseline reused these RQ1 results. The separately trained RQ2 input-only and combined models both selected checkpoints by highest validation macro-F1. For RQ3, both architectures used the highest macro-F1 on the source-domain validation set, excluding general writing. The different RQ1 selection criteria limit how confidently the performance difference can be attributed solely to architecture.

Table 6. Selected checkpoint epochs by experiment

Experiment	CNN selected epoch	LSTM selected epoch
RQ1: Output-only	9	5
RQ2: Input-only	5	9
RQ2: Combined	7	9
RQ3: Cross-domain	6	12

The notebooks save supporting artifacts such as training histories, metrics, predictions, confusion matrices, and model checkpoints to their configured output locations. Saved notebook outputs retain the reported results; a complete checkpoint and prediction archive is not included in the repository reviewed for this report.

The project was implemented in Python using PyTorch [4], with experiments executed in Google Colab. The saved CNN and LSTM training outputs record PyTorch 2.11.0+cu130 with CUDA available. Complete original Python, NumPy, pandas, scikit-learn, GPU-model, and driver versions were not retained in the submitted outputs, limiting exact environment reconstruction.

Our shared GitHub repository contains three primary experimental notebooks: a CNN notebook for the convolutional classification experiments, an LSTM notebook for the recurrent classification experiments, and a separate RQ4 notebook for linguistic and writing-style analysis. The repository also contains the shared dataset files and supporting project documentation. This organization separates neural-network training and evaluation from the descriptive analysis of LLM-generated responses.

The repository is available at:

https://github.com/palashDoshi1/CMPSC-448-Midterm-Project

Together, these materials document the implementation and evaluation procedures used in our experiments.

5. Experimental Results and Analysis

We evaluated the CNN and LSTM models across four research questions to investigate whether large language models can be identified from their generated responses. Our experiments examined overall classification performance, the influence of user prompts, generalization to an unseen domain, and linguistic characteristics that distinguish GPT, Claude, and Gemini.

We primarily evaluated classification performance using accuracy, macro-F1 scores, and confusion matrices. Training and validation curves were also examined to understand model learning behavior and potential overfitting.

The following sections present the experimental findings and compare the two architectures.

5.1 RQ1 - Can We Identify Which LLM Generated a Response?

Our first research question investigated whether the generated response alone contains enough information to identify its source LLM. Both CNN and LSTM classifiers were trained to distinguish among GPT, Claude, and Gemini using only the generated text, without including the original user prompt.

The experiment used the primary dataset split described in Section 3.4, consisting of 378 training responses, 81 validation responses, and 81 test responses. The three classes were equally represented in the test set, with 27 responses per model.

5.1.1 CNN Classification Performance

The CNN achieved **82.72% test accuracy** and a **macro-F1 score of 0.8293**, correctly classifying 67 of the 81 test responses.

Table 7. CNN RQ1 class-level classification metrics

Model	Precision	Recall	F1	Support
GPT	0.7419	0.8519	0.7931	27
Claude	0.9565	0.8148	0.8800	27
Gemini	0.8148	0.8148	0.8148	27
Macro average	0.8378	0.8272	0.8293	81

The macro-F1 is the unweighted mean of the three class F1 scores: (0.7931034483 + 0.8800000000 + 0.8148148148) / 3 = 0.8293060877, which rounds to 0.8293. Each class contains 27 test responses.

These results indicate that the CNN successfully learned patterns that distinguish responses generated by the three language models. The convolutional architecture was able to identify local textual patterns that were useful for classification.

The CNN achieved its best validation performance at epoch 9. Its training curves show that training accuracy continued to improve, while validation performance fluctuated, suggesting some overfitting as training progressed. Early stopping helped select a checkpoint based on validation performance rather than the final training epoch.

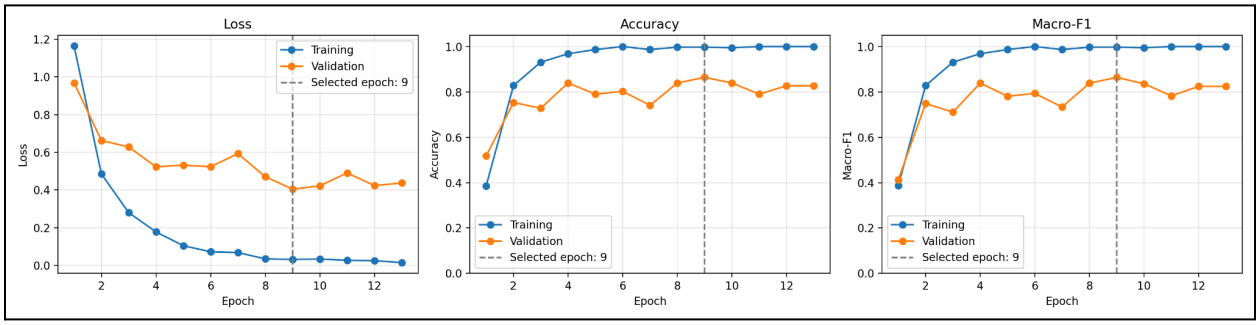


Figure 1. CNN training and validation performance for RQ1.

5.1.2 LSTM Classification Performance

The LSTM achieved **71.60% test accuracy** and a **macro-F1 score of 0.7141**.

Although the LSTM performed substantially better than the 33.33% chance-level baseline, its classification performance was lower than that of the CNN.

The LSTM achieved its best checkpoint at epoch 5. Its training and validation curves provide additional insight into how the model learned during training and whether performance improved consistently across epochs.

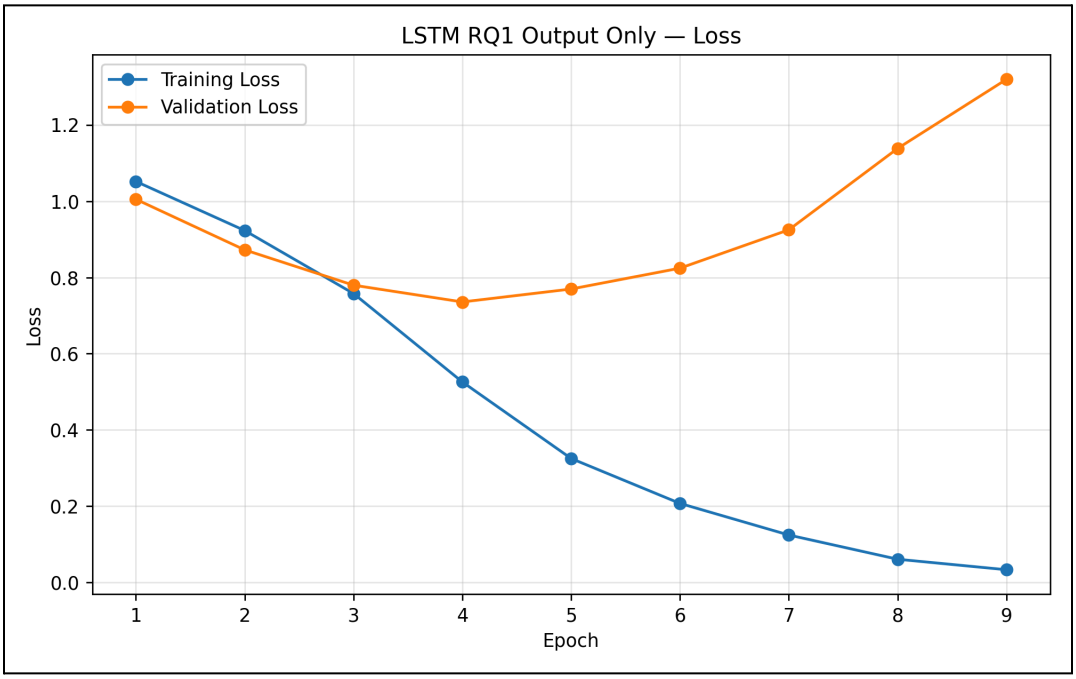


Figure 2. LSTM training and validation loss for RQ1.

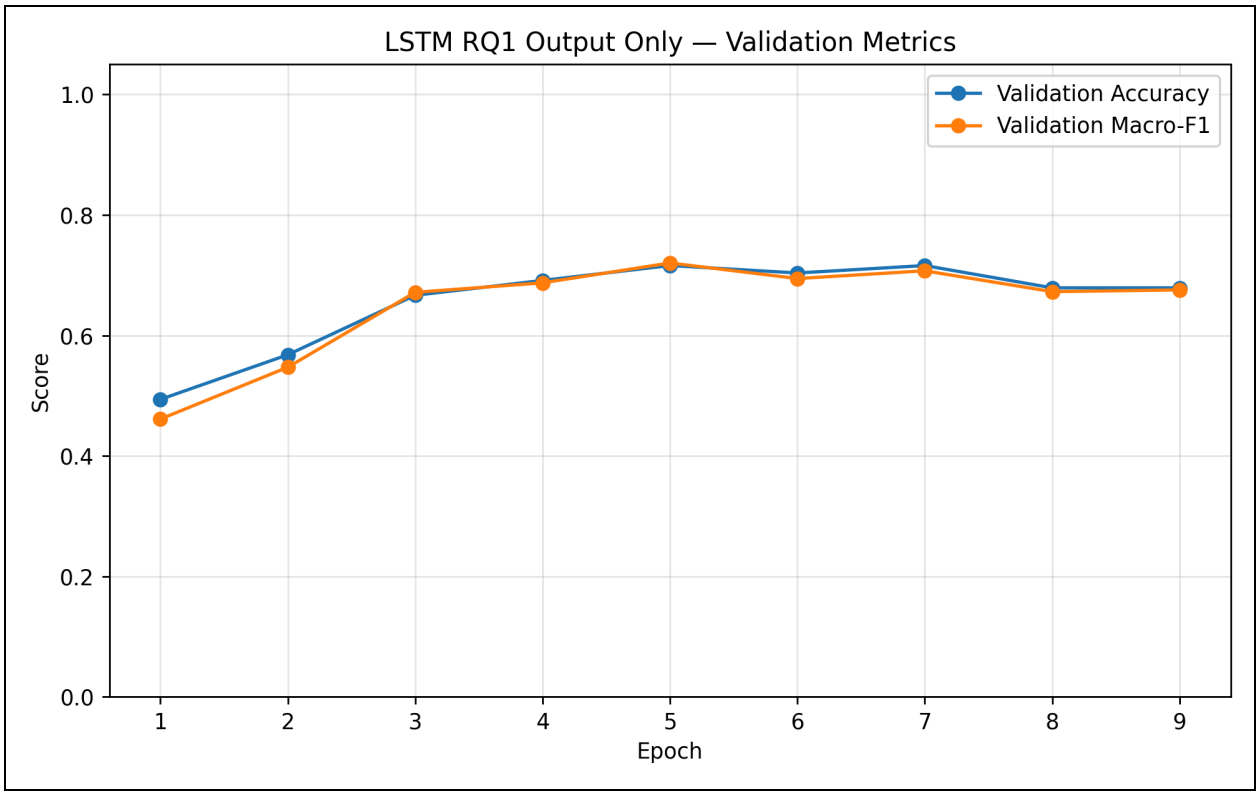


Figure 3. LSTM validation accuracy and macro-F1 for RQ1.

The LSTM confusion matrix provides a more detailed view of the model's classification performance across GPT, Claude, and Gemini.

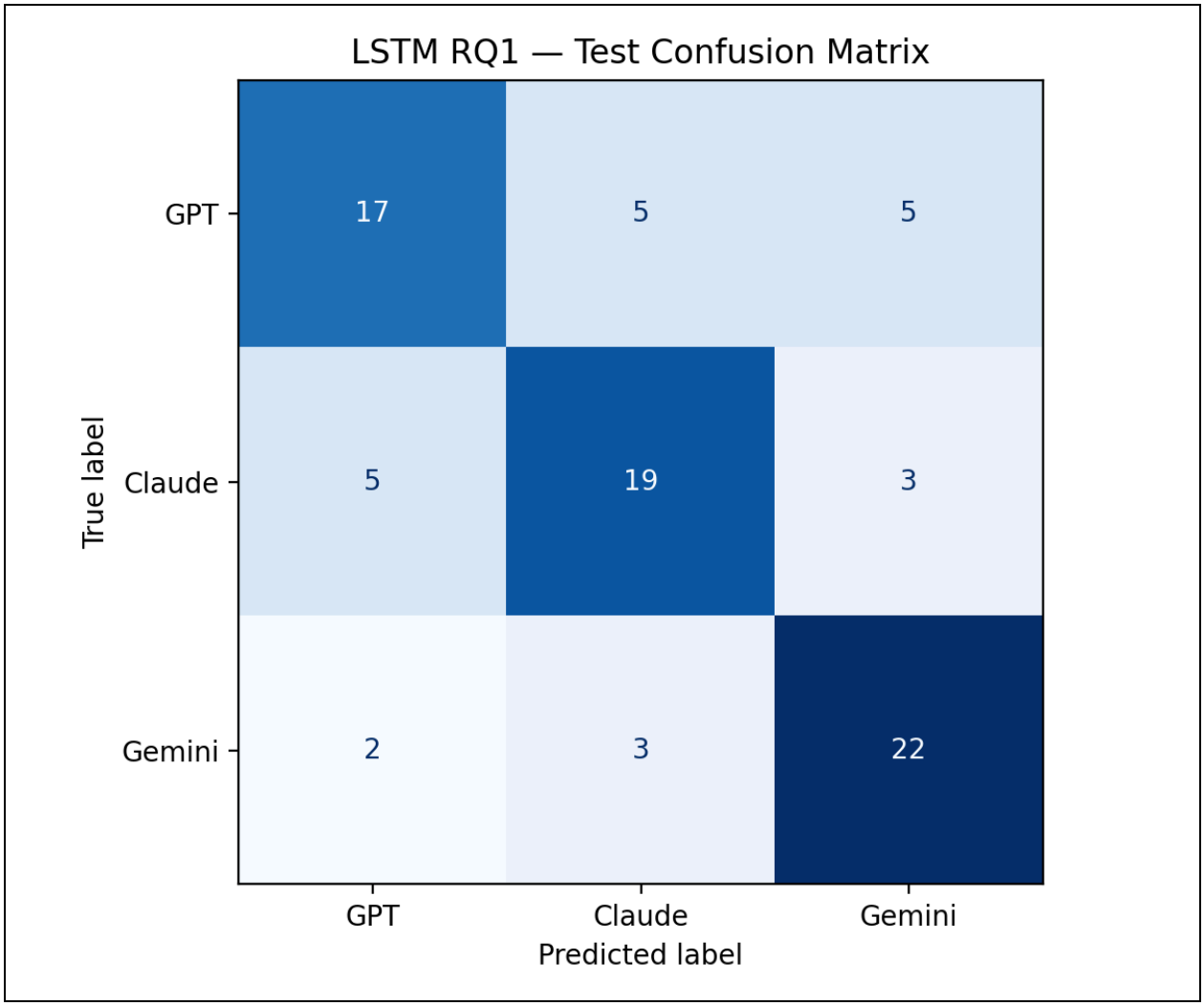


Figure 4. LSTM confusion matrix for output-only classification.

5.1.3 CNN and LSTM Comparison

Table 8. RQ1 classification performance

Model	Test accuracy	Macro-F1
CNN	**82.72%**	**0.8293**
LSTM	71.60%	0.7141
Chance baseline	33.33%	-

The CNN outperformed the LSTM by approximately 11.11 percentage points in accuracy and 0.1152 in macro-F1. This observed difference is descriptive; its statistical significance was not evaluated.

These findings suggest that local textual patterns captured by convolutional filters were particularly useful for identifying LLM authorship in our dataset. However, the results do not establish that CNNs are universally better than LSTMs, since the comparison is limited to our dataset, architectures, and experimental settings.

Overall, RQ1 demonstrates that responses generated by GPT, Claude, and Gemini contain distinguishable characteristics that neural-network classifiers can learn.

5.2 RQ2 - Does the User's Original Prompt Help Identify the LLM?

Our second research question investigated whether including the original user prompt improves the classification of LLM-generated responses.

We evaluated three configurations:

1. **Input-only:** The classifier receives only the original user prompt.
2. **Output-only:** The classifier receives only the generated response.
3. **Combined input/output:** The classifier receives both the original prompt and generated response.

This comparison was important because all three language models responded to the same prompts. Therefore, the prompt itself should not provide information about which model generated a particular response.

5.2.1 Input-Only Classification

With identical prompt inputs paired with all three labels, deterministic predictions yield exactly one correct classification per prompt group, giving 33.33% accuracy. Each of the 27 test prompts appears once with a GPT label, once with a Claude label, and once with a Gemini label. A deterministic classifier predicts the same class for all three copies, so it gets exactly 27 of 81 examples correct.

The value 33.33% is also the balanced three-class chance baseline, but the predictions themselves need not be random. Macro-F1 can differ because the models distribute their predictions differently across classes: the CNN achieved 0.3078 and the LSTM achieved 0.3281.

The CNN input-only confusion matrix illustrates this limitation.

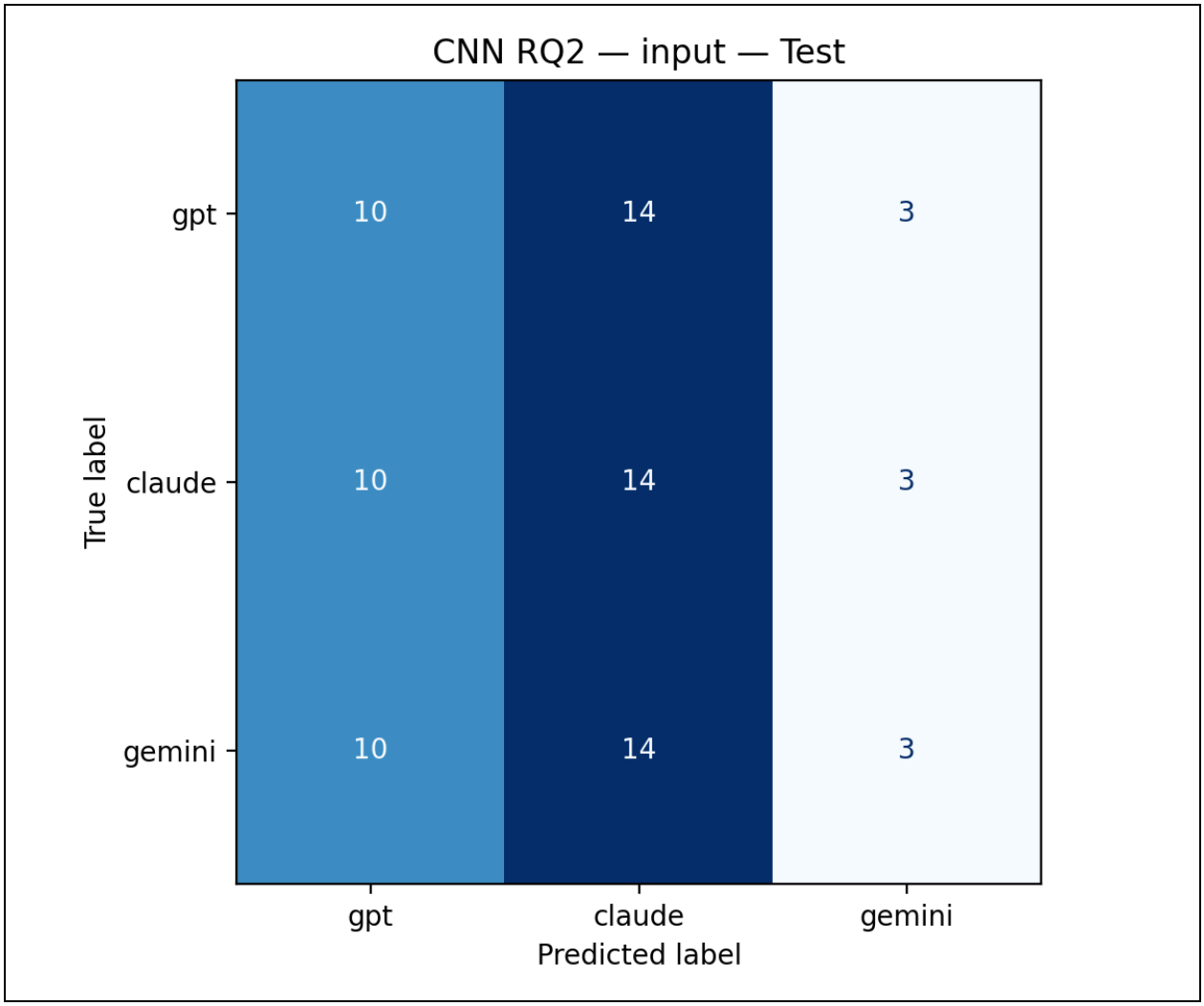


Figure 5. CNN confusion matrix for input-only classification.

Similarly, the LSTM confusion matrix shows its predictions when only the original prompt was available.

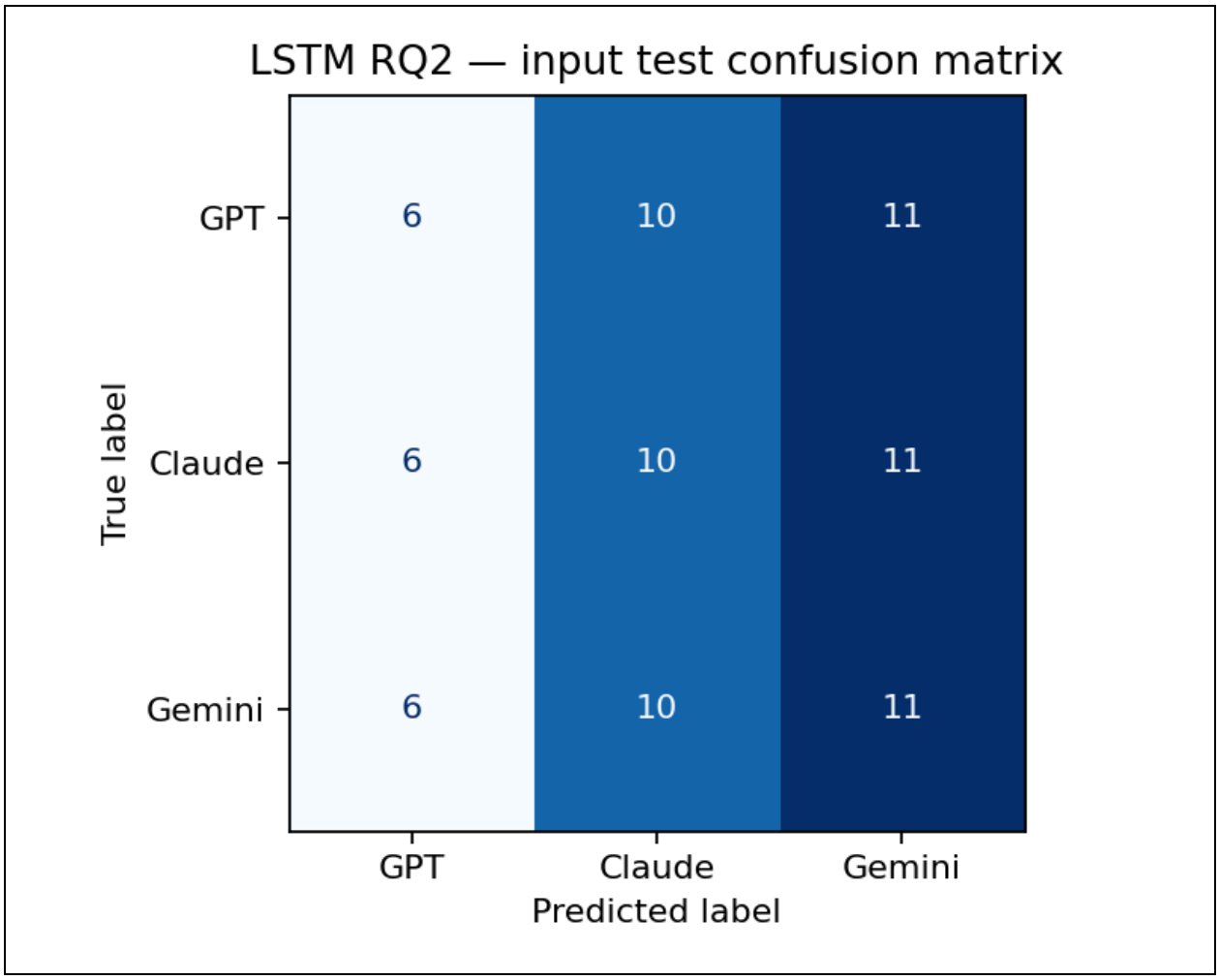


Figure 6. LSTM confusion matrix for input-only classification.

5.2.2 Combined Input/Output Classification

Next, we investigated whether combining the original prompt with the generated response improved classification performance.

The CNN achieved 77.78% accuracy and a macro-F1 score of 0.7730 in the combined-input experiment. This was lower than its output-only performance of 82.72% accuracy and 0.8293 macro-F1.

The LSTM achieved **53.09% accuracy** and **0.5294 macro-F1** in the combined-input experiment.

This was lower than its output-only performance of 71.60% accuracy.

Combined input performed worse under the tested token allocation. This comparison does not isolate the effect of adding the prompt because combined input also reduced the maximum response length from 512 to 383 tokens. Therefore, the lower scores cannot be attributed specifically to unhelpful prompt information.

The following confusion matrices illustrate the combined-input classification results.

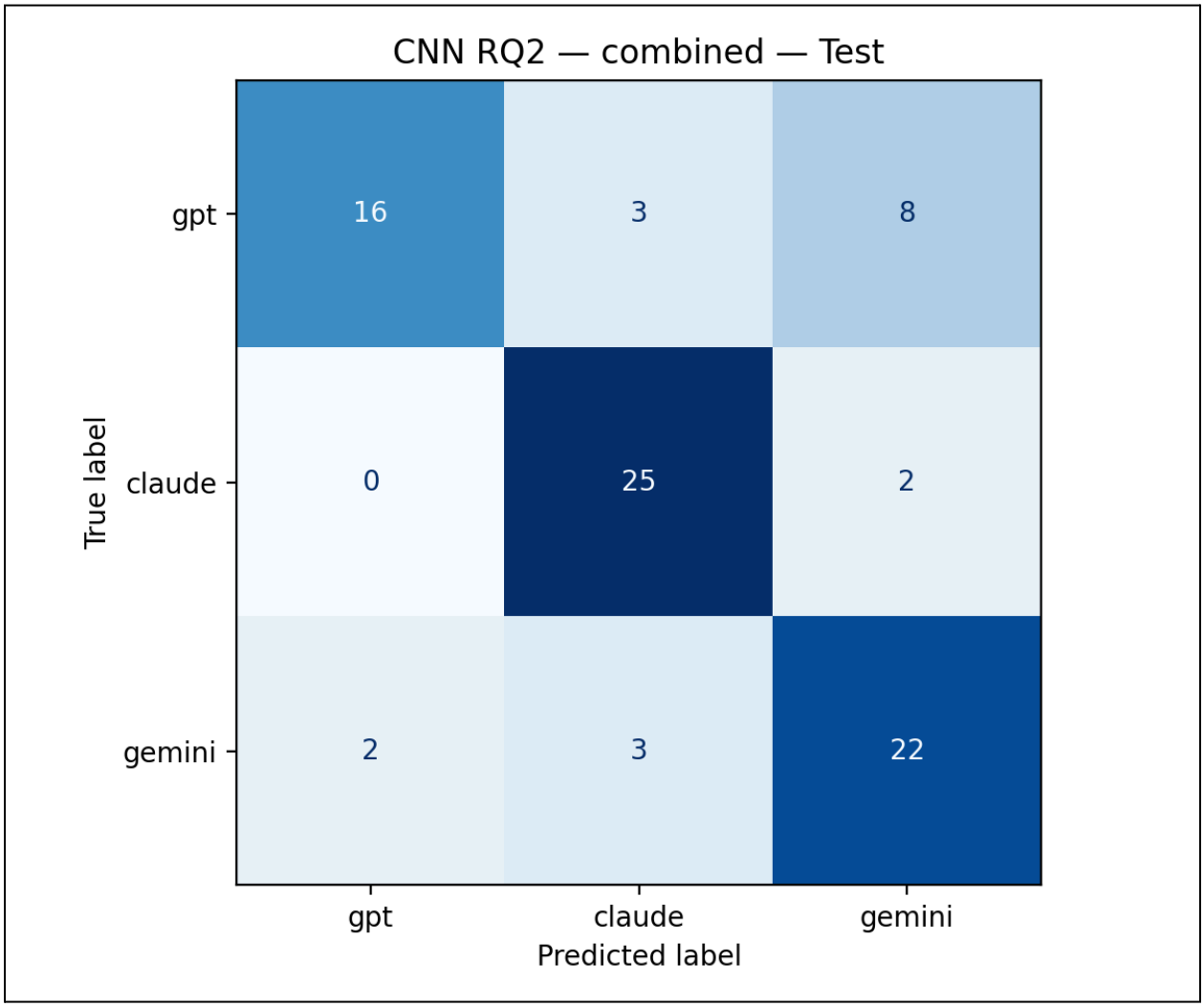


Figure 7. CNN confusion matrix for combined input/output classification.

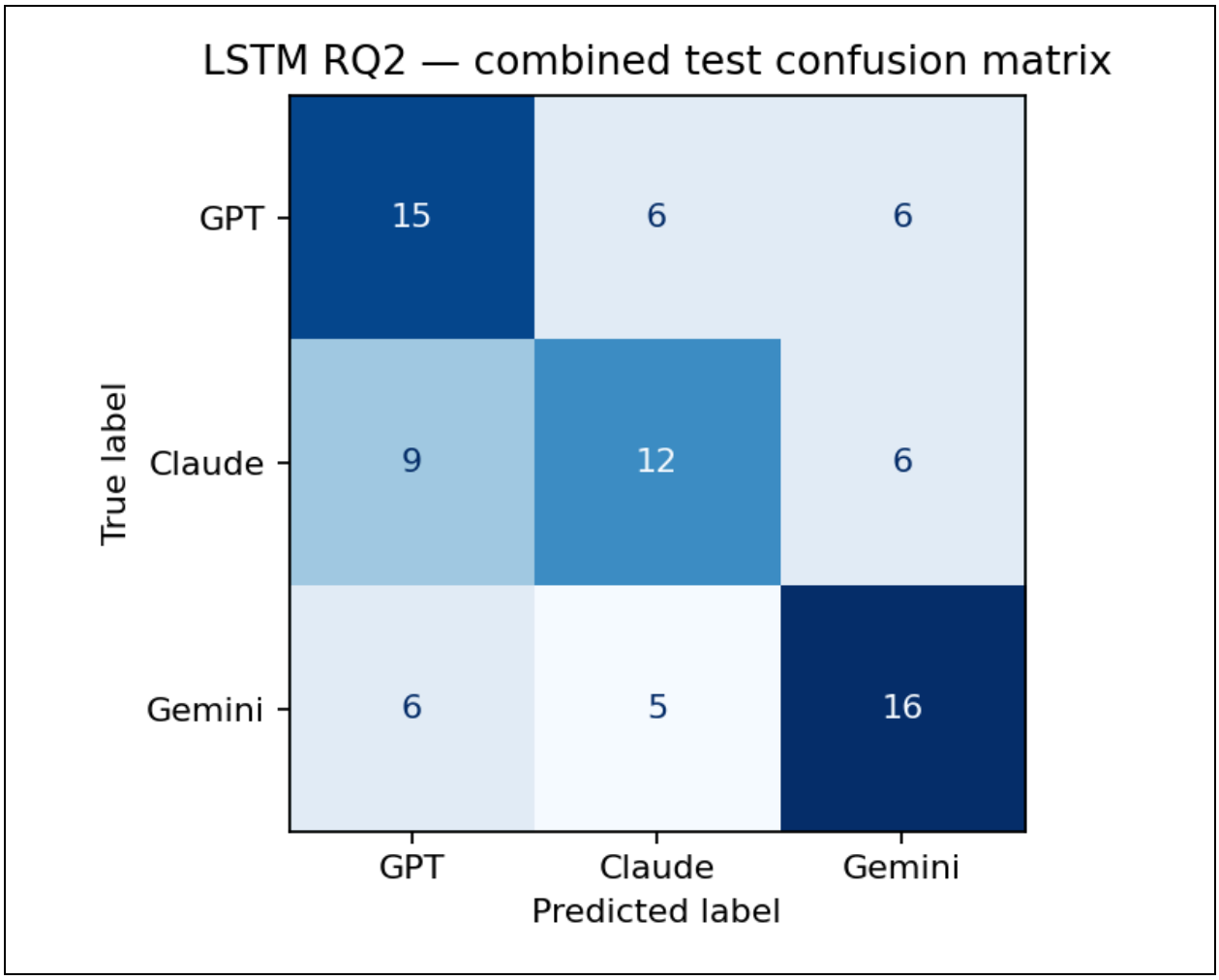


Figure 8. LSTM confusion matrix for combined input/output classification.

5.2.3 Interpretation of RQ2

Output-only classification achieved the highest observed accuracy for both architectures among the tested input configurations. Input-only accuracy was structurally fixed at one-third because each prompt was paired with all three labels, as explained in Section 5.2.1.

The combined-input comparison does not isolate the effect of prompt inclusion. Combined sequences retain at most 383 response tokens, compared with 512 for output-only input, and use a separately constructed training vocabulary. In addition, the CNN output-only baseline selected its checkpoint by validation loss, whereas the CNN combined model used validation macro-F1.

A controlled follow-up should use the same response-token budget, a shared training-derived vocabulary, and the same checkpoint-selection rule when comparing response-only and prompt-plus-response inputs. No such controlled experiment was performed here.

5.3 RQ3 - Do LLM Fingerprints Generalize Across Different Domains?

Our third research question investigated whether models trained on responses from familiar subject categories could identify the generating LLM in a previously unseen category.

For this experiment, we excluded the entire general-writing category from training and validation.

The CNN and LSTM were trained on responses from the remaining five categories: computer science and software, finance and banking, mathematics and logic, sports, and stocks and trading.

We then evaluated the classifiers on **all 90 general-writing responses**, consisting of 30 responses each from GPT, Claude, and Gemini.

This experiment tested whether the classifiers learned transferable writing characteristics rather than relying exclusively on patterns associated with particular subject categories.

5.3.1 CNN Cross-Domain Performance

The CNN achieved **89.39% accuracy** and **0.8932 macro-F1** on its held-out source-domain test set.

However, when evaluated on the unseen general-writing category, its accuracy decreased to **72.22%**, with a macro-F1 score of **0.7161**.

This represents a decrease of **17.17 percentage points** in accuracy.

The source-domain confusion matrix demonstrates the CNN's classification performance on responses from familiar categories.

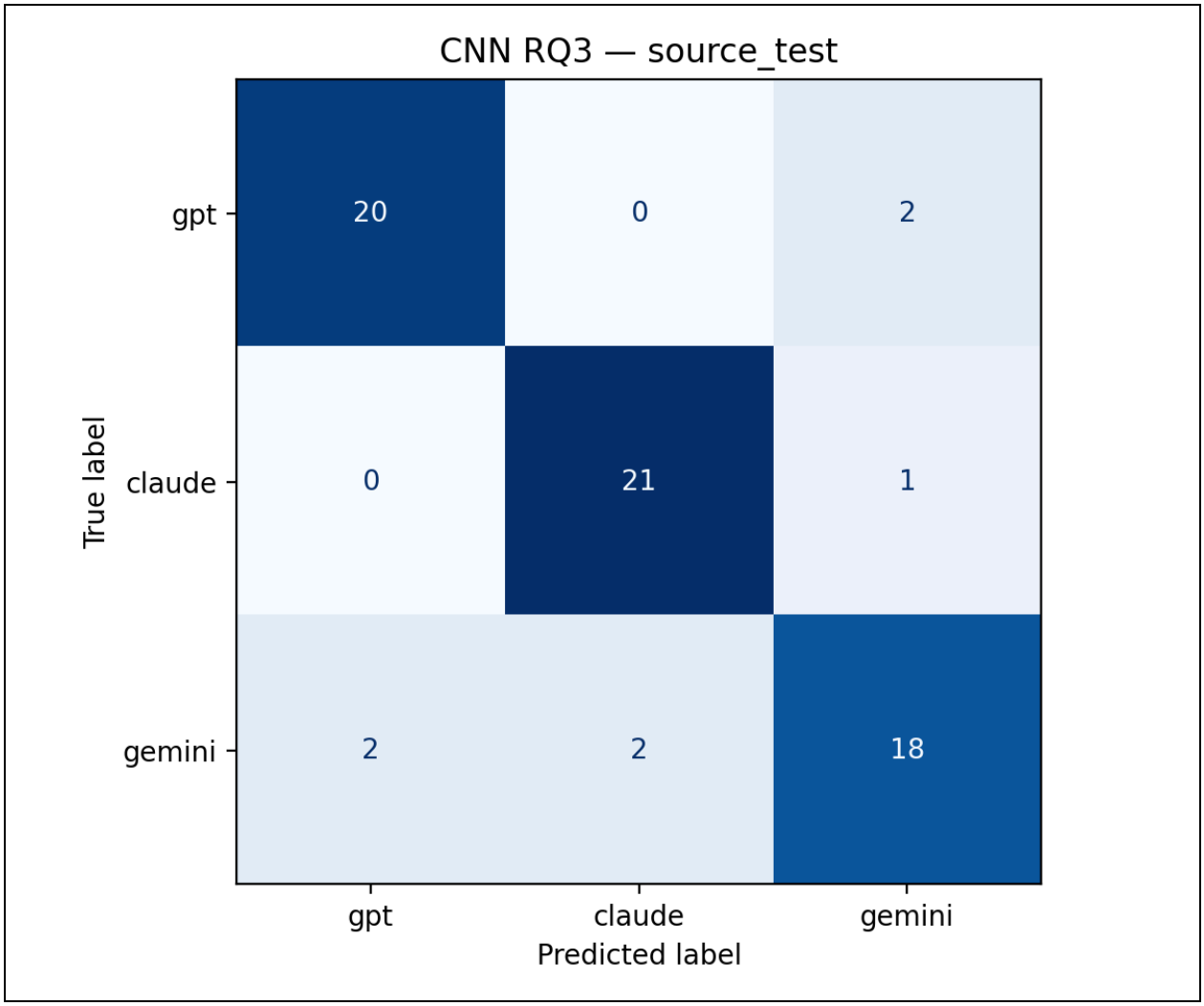


Figure 9. CNN confusion matrix for the held-out source-domain test set.

The unseen-domain confusion matrix shows how classification performance changed when the CNN encountered general-writing responses.

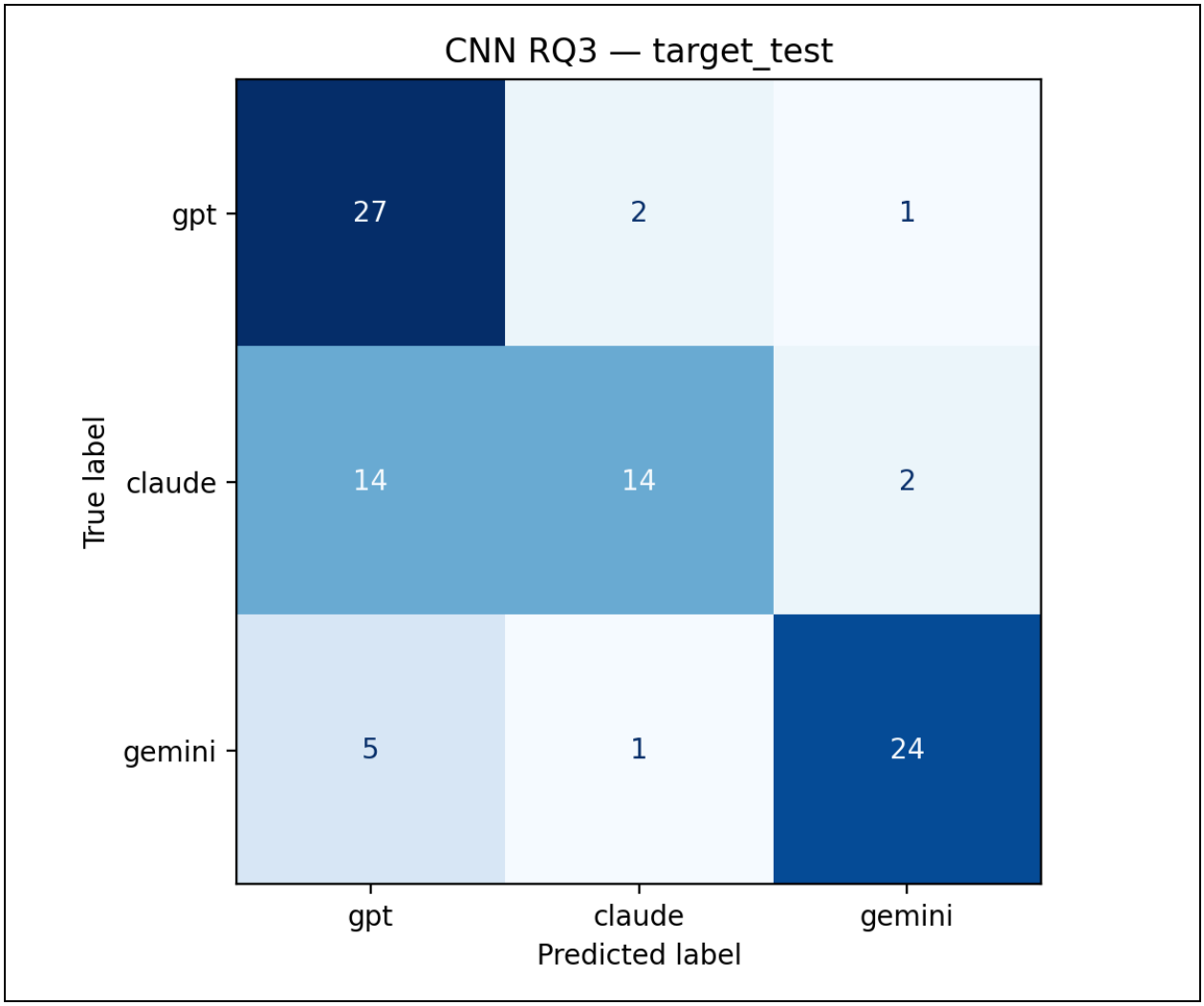


Figure 10. CNN confusion matrix for the unseen general-writing domain.

The CNN correctly classified 65 of the 90 unseen writing responses.

However, classification performance varied among the three language models. GPT recall reached 90%, Gemini recall reached 80%, and Claude recall was 46.67%.

In particular, 14 Claude responses were incorrectly classified as GPT. This suggests that Claude's writing characteristics were more difficult for the CNN to distinguish from GPT in the held-out general-writing category.

5.3.2 LSTM Cross-Domain Performance

The LSTM achieved 51.52% accuracy and 0.4890 macro-F1 on its held-out source-domain test set.

On the unseen general-writing category, it achieved **56.67% accuracy** and **0.5429 macro-F1**.

Unlike the CNN, the LSTM did not experience an accuracy decrease in this particular comparison. Its accuracy increased by 5.15 percentage points, although its overall classification performance remained relatively weak.

The following figures illustrate the LSTM's cross-domain classification results.

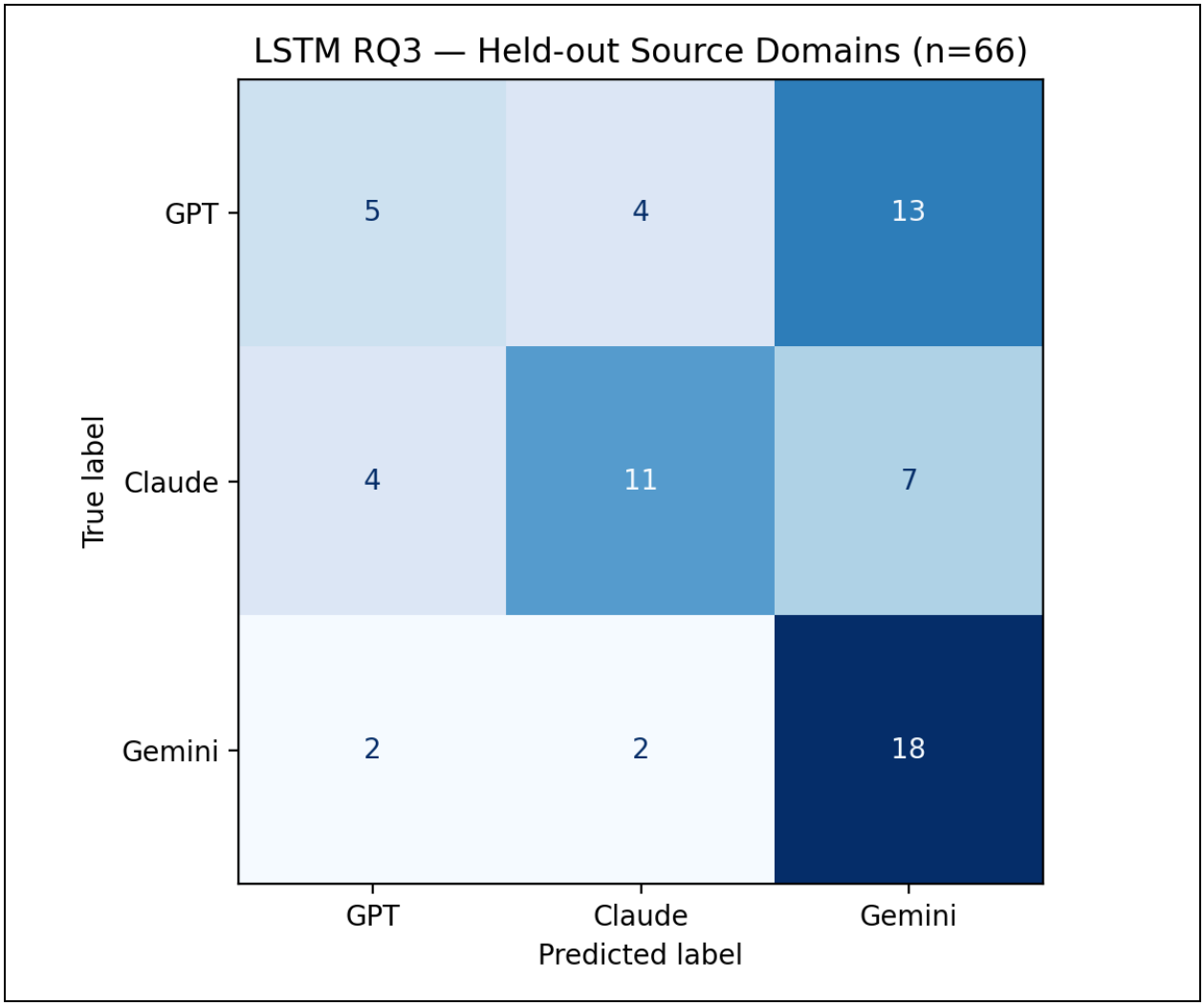


Figure 11. LSTM confusion matrix for the held-out source-domain test set.

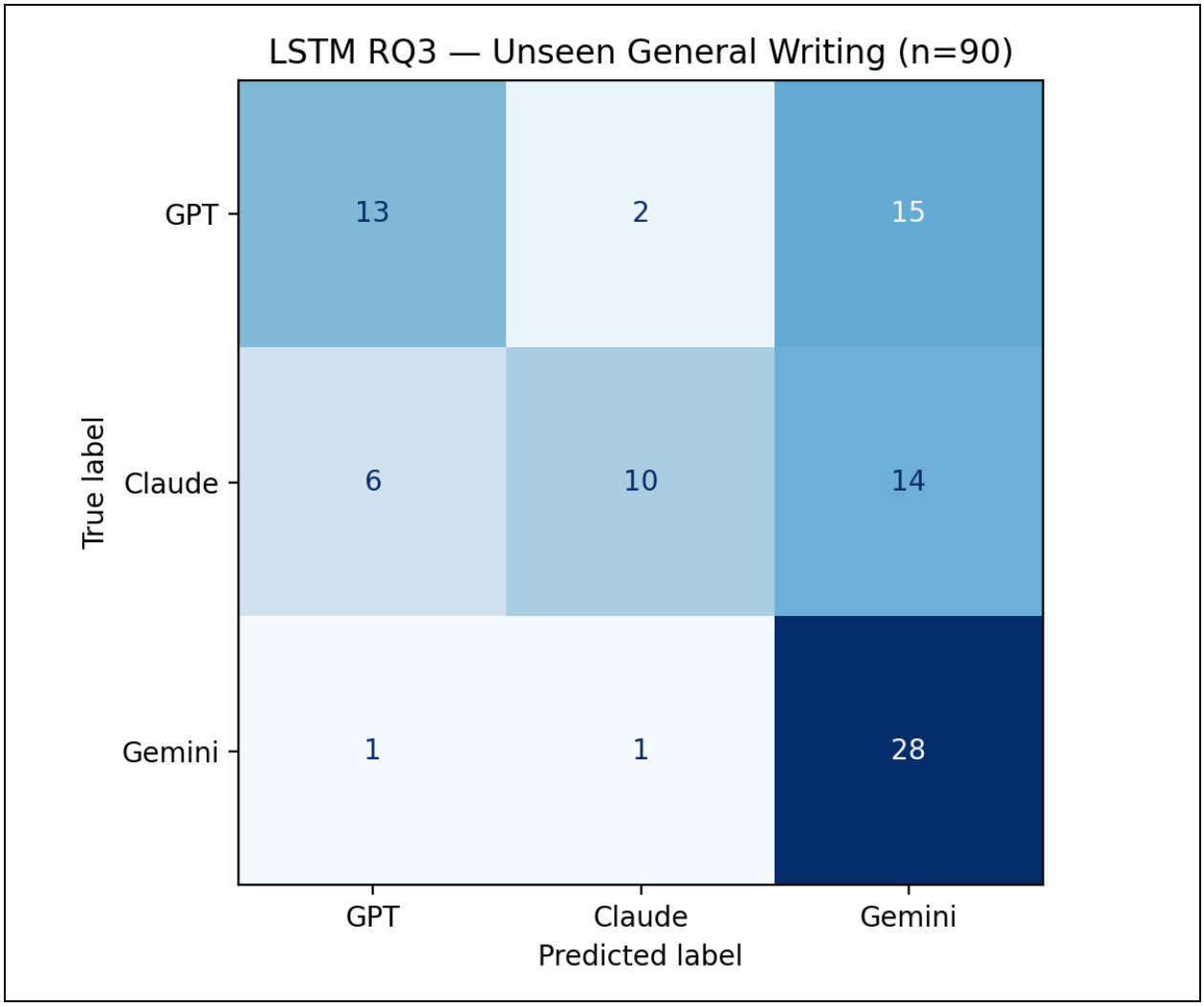


Figure 12. LSTM confusion matrix for the unseen general-writing domain.

The LSTM exhibited uneven classification performance across the three models. Gemini recall reached approximately 93%, while GPT and Claude recall were approximately 43% and 33%, respectively.

These results indicate that the LSTM was more successful at identifying Gemini responses but frequently misclassified responses generated by GPT and Claude.

5.3.3 CNN and LSTM Cross-Domain Comparison

Table 9. RQ3 cross-domain classification performance

Metric	CNN	LSTM
Source-domain test accuracy	89.39%	51.52%
Unseen-domain accuracy	**72.22%**	56.67%
Unseen-domain macro-F1	**0.7161**	0.5429
Source-to-target accuracy change	−17.17 pp	+5.15 pp

On the unseen general-writing domain, the CNN outperformed the LSTM by 15.56 percentage points in accuracy. This observed difference is descriptive; its statistical significance was not evaluated.

The CNN also achieved a higher macro-F1 score, indicating stronger classification performance across the three classes.

Both architectures used identical prompt assignments, not merely equal partition sizes. Each notebook filtered general-writing rows from the same shared train.csv, validation.csv, and test.csv files, preserving 315 source-domain training responses, 69 validation responses, and 66 source-domain test responses. The prompt_id fields retain these assignments. Both models were evaluated on the same 90 general-writing responses. Differences in architecture and training should still be considered when interpreting the results.

Overall, RQ3 demonstrates that some LLM-specific characteristics can transfer across subject domains, but classification performance depends on the architecture and evaluation domain.

5.4 RQ4 - What Linguistic and Structural Characteristics Distinguish Different LLMs?

Our fourth research question investigated the linguistic and structural characteristics of responses generated by GPT, Claude, and Gemini.

Unlike RQ1–RQ3, which focused on supervised classification performance using CNN and LSTM models, RQ4 was implemented in a separate Jupyter notebook dedicated to descriptive linguistic analysis. We analyzed measurable writing characteristics across all 540 responses to investigate differences in response length, lexical diversity, punctuation, and formatting among GPT, Claude, and Gemini. This experiment did not involve training an additional neural-network classifier.

We considered characteristics such as response length, vocabulary usage, lexical diversity, and formatting patterns.

RQ4 measurements were calculated from the complete, untruncated response strings. Code and Markdown were not removed before analysis. Words were counted using Python's Unicode-aware regular expression `r"\b\w+\b"`, which matches sequences of alphanumeric characters and underscores. Consequently, numbers and code identifiers also contribute to the word count.

Lexical diversity was measured using the moving-average type–token ratio introduced by Covington and McFall [5], with a window size of 50 words. The extracted words were lowercased, and the number of distinct words in each window was divided by 50. Windows advanced one word at a time, and their ratios were averaged to obtain one MATTR-50 value per response. Responses containing fewer than 50 words received no MATTR-50 value and were excluded only from that measurement. This produced 179 eligible GPT responses and 180 eligible responses each for Claude and Gemini. Model-level MATTR-50 values are unweighted averages of the eligible response-level values.

Punctuation and symbol counts used `r"[^\w\s]"`, which matches individual characters that are neither word characters nor whitespace. This includes punctuation, Markdown symbols and code operators; underscores are excluded because they are word characters. For each response, the rate was calculated as 100 times the punctuation/symbol count divided by its

word count. The reported model-level rate is the unweighted mean of these per-response rates, rather than a ratio calculated from pooled totals.

Formatting markers were detected using simple rules rather than a Markdown parser. Each response was divided with splitlines(); heading lines matched r"^\s{0,3}#{1,6}\s+", and list lines matched r"^\s*(?:[-*+]|\d+[.])\s+". Code-fence markers were detected by the substring "```" or "~~~" anywhere in the response, and bold markers by "**" or "__". These checks did not require balanced opening and closing markers and could also match text inside code. Each percentage is 100 times the number of responses containing at least one matching marker divided by 180 for that model; it is not a count of individual markers.

5.4.1 Writing-Style Distributions

Table 10 and Figure 13 summarize the response-level measurements. The distributions describe the stored responses and were not used to train or tune the neural-network classifiers.

Table 10. Linguistic and formatting characteristics of GPT, Claude, and Gemini

Measurement	GPT	Claude	Gemini
Mean response length (words)	459.27	629.71	328.78
Mean lexical diversity (MATTR-50)	0.8592	0.8259	0.8625
Punctuation/symbols per 100 words	26.53	26.00	41.25
Responses with heading markers	30.56%	0.56%	39.44%
Responses with list markers	19.44%	61.67%	33.33%
Responses with code fences	5.00%	0.00%	23.89%
Responses with bold markers	25.56%	19.44%	46.67%

The quantitative analysis revealed differences in response length, lexical diversity, punctuation, and formatting across the three LLM families. Claude generated the longest responses and used list markers most frequently, while Gemini produced the shortest responses and showed the highest frequencies of heading markers, code fences, and bold formatting markers. GPT generally occupied an intermediate position in response length. These descriptive differences provide potential indicators of model-specific writing behavior, although they do not establish a causal relationship with classifier predictions.

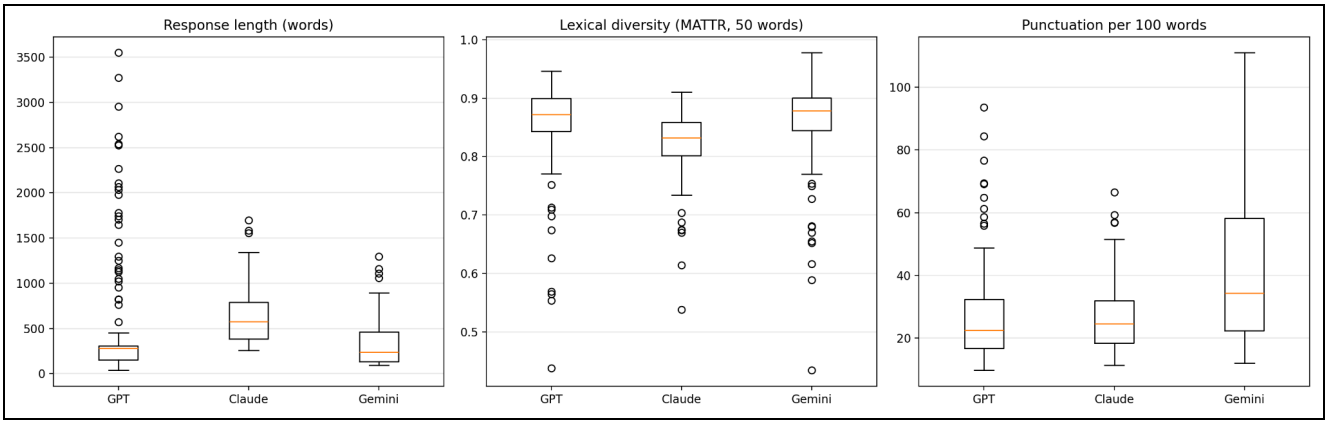


Figure 13. Distribution of selected writing-style characteristics across GPT, Claude, and Gemini.

The figure illustrates variation in writing characteristics across the three models. The distributions overlap, so these features do not provide a simple, perfectly separating rule. We did not evaluate classification reliability using individual features alone.

5.4.2 Formatting Characteristics

We also investigated formatting patterns in the generated responses.

Formatting can provide useful structural information because language models may differ in how frequently they use headings, lists, or other organizational elements.

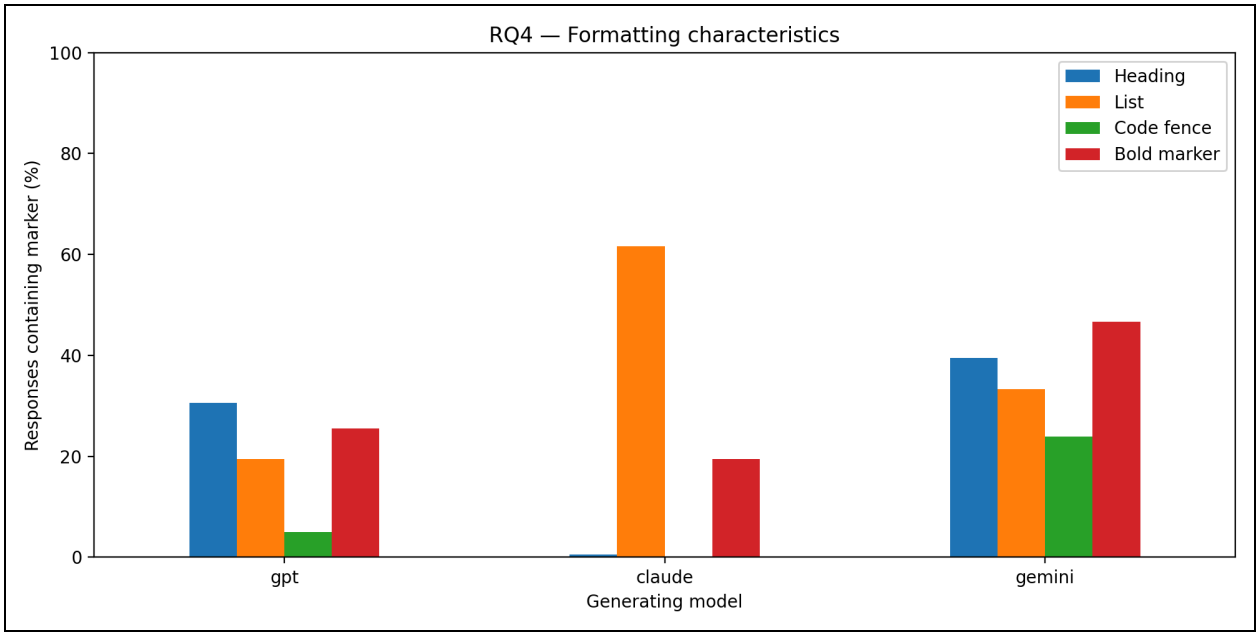


Figure 14. Comparison of formatting characteristics across GPT, Claude, and Gemini.

The observed differences in formatting suggest that response structure may contribute to LLM authorship identification.

However, these characteristics should be interpreted as descriptive patterns rather than definitive evidence of model identity. Formatting choices may also depend on the original prompt and the type of response requested.

5.4.3 Interpretation of RQ4

The writing-style analysis provides a possible explanation for why the CNN and LSTM were able to classify generated responses more accurately than random guessing.

Features such as vocabulary usage, response organization, and lexical diversity may contribute to the distinguishable patterns learned by neural-network classifiers.

Nevertheless, RQ4 does not directly establish which features the trained CNN and LSTM relied upon when making predictions.

A more detailed feature-attribution analysis would be required to identify the exact characteristics influencing individual model predictions.

Overall, RQ4 supports the conclusion that LLM-generated responses exhibit measurable linguistic and structural variation, which may help explain the success of authorship classification.

5.5 Overall Comparison of CNN and LSTM Performance

Table 11 summarizes the main classification results across RQ1, RQ2, and RQ3.

Table 11. Summary of CNN and LSTM classification results

Experiment	CNN accuracy	LSTM accuracy
RQ1: Output-only	**82.72%**	71.60%
RQ2: Input-only	33.33%	33.33%
RQ2: Combined input/output	**77.78%**	53.09%
RQ3: Unseen general writing	**72.22%**	56.67%

Across the experiments with confirmed results, the CNN achieved stronger classification performance than the LSTM on the primary output-only task and the unseen general-writing domain.

The input-only experiment demonstrated that original prompts alone did not provide useful authorship information.

The cross-domain experiments further showed that classification performance can vary substantially when models encounter responses from a category excluded during training.

These findings suggest that CNN-based classification was particularly effective for this dataset, while also highlighting the importance of careful evaluation across different input configurations and subject domains.

5.6 Discussion of Experimental Findings

Our experiments demonstrated that CNN and LSTM classifiers could identify responses generated by GPT, Claude, and Gemini with accuracy above the 33.33% chance baseline. The CNN achieved stronger performance than the LSTM on the primary output-only classification task, suggesting that local textual patterns were useful for distinguishing the three LLM families in our dataset.

Output-only input produced the highest observed scores among the RQ2 configurations. Combined input performed worse under the tested token allocation, which reduced the response budget from 512 to 383 tokens and therefore did not isolate prompt inclusion. In RQ3, the CNN had higher unseen-domain accuracy than the LSTM but lower accuracy than on its own source-domain test set; the LSTM's unseen-domain accuracy increased relative to its source-domain result. RQ4 identified descriptive differences in response length, lexical diversity, punctuation, and formatting.

Overall, these findings support the feasibility of LLM authorship classification under our experimental conditions. A more detailed interpretation of these results is presented in Section 6.

5.7 Limitations of the Experiments

Our experiments have several limitations. The dataset contained 540 responses from three LLM families, limiting generalization to other models and tasks. The primary experiments used one prompt-grouped split and one recorded training seed per configuration. Reported performance differences are descriptive and were not established as statistically significant. Responses sharing a prompt are related observations, so future uncertainty estimates should account for prompt groups.

Additionally, the cross-domain experiment evaluated only general writing as an unseen category, rather than testing all six categories individually. The writing-style analysis was descriptive and did not establish which linguistic or structural features directly influenced classifier predictions.

These limitations define the scope of our findings and are discussed in greater detail in Section 6.5.

6. Discussion and Limitations

6.1 Interpretation of Key Findings

The experimental results demonstrate that neural-network classifiers can identify the generating large language model from its textual responses with accuracy substantially above the three-class chance baseline of 33.33%. Both the Convolutional Neural Network (CNN) and Long Short-Term Memory (LSTM) models learned patterns that helped distinguish responses generated by GPT, Claude, and Gemini.

For the primary response-only classification task (RQ1), the CNN achieved 82.72% test accuracy and a macro-F1 score of 0.8293, while the LSTM achieved 71.60% accuracy and a macro-F1 score of 0.7141. These results suggest that the generated responses contained identifiable characteristics that the models could learn from the available training data.

The CNN achieved higher accuracy in the output-only, combined-input, source-domain, and unseen-domain evaluations; both models tied in input-only accuracy. The LSTM had the higher input-only macro-F1 (0.3281 versus 0.3078). These results are specific to the tested configurations and do not establish that CNN architectures are universally superior to LSTM architectures for LLM authorship classification.

Performance also depended on the input configuration and evaluation domain. Output-only input had the strongest observed scores under the tested token allocations; the combined configuration retained fewer response tokens, so this comparison does not isolate the value of adding the prompt. The cross-domain experiment showed different source-to-target behavior for the two architectures.

Overall, the findings support the feasibility of LLM authorship classification while demonstrating that performance is influenced by model architecture, input representation, and the characteristics of the evaluation dataset.

6.2 Comparison of CNN and LSTM Performance

The CNN and LSTM architectures approached the classification task differently. The CNN used convolutional filters to identify local patterns across token sequences, while the LSTM processed tokens sequentially to construct a representation of the text.

In the primary experiment, the CNN outperformed the LSTM by approximately 11.11 percentage points in test accuracy and had a higher macro-F1 score. These are descriptive differences from a single split and recorded run; their statistical significance was not evaluated.

One possible explanation is that distinguishing GPT, Claude, and Gemini may depend partly on recurring local patterns, such as particular word combinations, punctuation sequences, and formatting-related tokens. The CNN architecture is designed to detect such patterns through filters operating over short token windows.

The LSTM, in contrast, was designed to capture sequential relationships through its recurrent structure. Although this approach can represent dependencies across text, its effectiveness depends on factors such as the amount of training data, sequence length, model capacity, and optimization.

The relatively small dataset of 540 responses may have limited the ability of the LSTM to learn sufficiently generalizable representations. However, the experiments do not isolate dataset size as the cause of the performance difference.

Another consideration is that the CNN and LSTM used different checkpoint-selection criteria in the primary experiment. The CNN selected its checkpoint using validation loss, while the LSTM used validation macro-F1. Although both approaches are valid, this difference limits how confidently the observed performance gap can be attributed solely to architecture.

Therefore, the results indicate that the CNN was more effective under our specific experimental conditions, rather than proving an inherent advantage of convolutional networks over recurrent networks.

6.3 Influence of Original Prompts on Classification

The second research question investigated whether including the original user prompt improved the ability to identify the generating LLM.

We evaluated three input configurations: input-only, output-only, and combined input/output.

Input-only accuracy was exactly 33.33% for both models because each test prompt appeared with all three labels. Deterministic predictions are identical across those three inputs and therefore produce one correct classification per prompt group. This equals the balanced chance baseline without requiring random predictions.

The output-only configuration achieved the strongest performance for both architectures. The CNN reached 82.72% accuracy, while the LSTM reached 71.60%.

When the original prompt was combined with the generated response, performance decreased to 77.78% for the CNN and 53.09% for the LSTM.

Combined input performed worse under the tested token allocation. This comparison does not isolate the effect of adding the prompt because combined input also reduced the maximum response length from 512 to 383 tokens. The reduction is imposed even when a prompt contains fewer than 128 tokens.

The input configurations also used separately constructed vocabularies, and the CNN output-only and combined runs used different checkpoint-selection criteria. An equal-response-budget comparison with a shared training-derived vocabulary and matching selection criteria would better isolate the contribution of the prompt. The current results establish only which tested configuration achieved the higher observed score.

6.4 Cross-Domain Generalization and Robustness

The third research question examined whether classifiers trained on several subject categories could identify the generating model when evaluated on a previously unseen category.

To investigate this, we excluded the general-writing category from training and validation and evaluated the models on all 90 general-writing responses.

The CNN achieved 72.22% accuracy and a macro-F1 score of 0.7161, while the LSTM achieved 56.67% accuracy and a macro-F1 score of 0.5429.

Both models performed above the 33.33% chance baseline, suggesting that some of the learned patterns remained useful when applied to a category excluded from training.

When comparing the source-domain and unseen-domain evaluations within RQ3, the CNN's accuracy decreased from 89.39% to 72.22%, a reduction of 17.17 percentage points. In contrast, the LSTM's accuracy increased from 51.52% to 56.67%, although its overall performance remained comparatively weak. These findings indicate that cross-domain performance varied between architectures and that patterns learned from the source categories did not transfer equally well across the two classifiers.

The CNN maintained stronger performance than the LSTM, outperforming it by approximately 15.56 percentage points on the held-out general-writing category.

The LSTM also showed uneven performance across classes in this experiment. Its reported recall was particularly high for Gemini but substantially lower for GPT and Claude, suggesting that it was more successful at recognizing one model family than the others.

An important limitation is that this experiment examined only one held-out category. Although the results provide evidence of partial cross-domain transfer, they do not establish that the classifiers would generalize equally well to every other unseen category.

A more comprehensive evaluation would require repeating the experiment with different held-out domains.

6.5 Limitations of the Study

Although our project produced meaningful experimental findings, several limitations should be considered when interpreting the results.

Dataset Size: Our dataset contained 540 responses generated from 180 original prompts. While this was sufficient to train and evaluate the selected neural-network architectures, it was relatively small compared with datasets commonly used for large-scale text-classification research. A larger dataset could provide more varied examples and support stronger conclusions about model generalization.

Limited Model Coverage: The study considered three LLM families: GPT, Claude, and Gemini. The results therefore cannot automatically be generalized to other language models. Additionally, differences between versions of the same LLM family were not independently evaluated.

Limited Subject Categories: The dataset covered six categories selected for this project. Although these categories included both technical and nontechnical tasks, they did not represent the full range of possible user prompts. Other domains, such as legal writing, medical explanations, or creative storytelling, may produce different classification behavior.

Response Characteristics: Classification may have been influenced by response length, formatting, vocabulary, and other observable properties. These characteristics can vary with the prompt, model version, and generation settings. Consequently, a classifier may learn patterns associated with the particular response-generation conditions rather than stable characteristics of an entire LLM family.

Architecture and Hyperparameter Choices: We evaluated one primary CNN architecture and one primary LSTM architecture. Different embedding sizes, hidden dimensions, convolutional filters, learning rates, or training strategies could produce different results. Therefore, our findings are specific to the implementations and configurations tested.

Evaluation Variability: The primary test set contained 81 responses from 27 prompt groups. The reported scores and gaps describe a single split and one recorded run per configuration; confidence intervals and significance tests were not calculated. The differences were therefore not established as statistically significant. Future paired comparisons should preserve

predictions from both architectures for the same examples and resample or analyze whole prompt groups rather than treating all responses as independent. Repeated training seeds and alternative prompt-grouped splits would also help estimate variability.

Collection and Reproducibility Records: Responses were manually copied from ordinary chat interactions. Generation settings, conversation-reset details, the original split-generation seed, and complete software-version information were not retained in the submitted artifacts. The saved response strings and predefined split assignments support replication of the analyses, but they do not fully reconstruct the original generation process or training environment.

Cross-Domain Evaluation: Our cross-domain experiment used general writing as the only held-out subject category. This allowed us to investigate transfer to one unseen domain but did not establish generalization across all possible domains.

These limitations do not invalidate the experimental results. Instead, they define the scope within which our conclusions should be interpreted.

6.6 Future Improvements and Research Directions

Several improvements could extend the current project and provide a deeper understanding of LLM authorship classification.

One important direction would be to expand the dataset by collecting more prompts and responses across a wider range of subjects. A larger and more diverse dataset could help determine whether the patterns learned by the classifiers remain consistent across different writing tasks.

Another improvement would be to evaluate additional model architectures. Transformer-based classifiers, such as BERT-based models, could provide an informative comparison with the CNN and LSTM approaches used in this study. Such experiments could help determine whether contextual representations improve authorship classification.

Future work could also investigate additional LLM families and multiple versions of the same model. This would help determine whether classifiers can distinguish between closely related model versions and whether their performance remains stable as language models evolve.

A more comprehensive cross-domain evaluation would involve holding out each subject category in turn. This leave-one-category-out approach would provide a broader understanding of which domains are easier or more difficult for authorship classification.

A controlled RQ2 follow-up should compare response-only and prompt-plus-response inputs with equal response-token budgets, a shared vocabulary built only from training text, and matching checkpoint-selection criteria. Further experiments could then examine response length, tokenization, vocabulary size, and sequence truncation separately.

Finally, more systematic linguistic analysis could be used to investigate the relationship between measurable writing characteristics and classifier predictions. Features such as sentence length, punctuation frequency, heading usage, list formatting, and lexical diversity could provide additional insight into the signals used for distinguishing LLM-generated responses.

Together, these improvements could strengthen the reliability, interpretability, and generalizability of future LLM authorship classification systems.

7. Conclusion and Future Work

7.1 Project Summary

This project investigated whether neural-network classifiers could identify the source of text generated by three large language model families: GPT, Claude, and Gemini. Our primary objective was to determine whether the responses produced by these models contained sufficiently distinctive textual characteristics to allow automated authorship classification.

To investigate this problem, we constructed an original and balanced dataset consisting of 540 generated responses collected from 180 prompts across six subject categories. Each prompt was submitted to all three LLM families, ensuring that the dataset contained an equal number of responses from each model.

We implemented and evaluated two neural-network architectures: a Convolutional Neural Network (CNN) and a Long Short-Term Memory (LSTM) network. Both models were trained to perform three-class classification using the generated text.

Our investigation extended beyond the primary classification task to examine the influence of original prompts, cross-domain generalization, and measurable writing characteristics. These experiments provided a broader understanding of the opportunities and limitations associated with identifying the source of LLM-generated content.

The results demonstrated that both neural-network architectures could achieve classification performance substantially above the three-class chance baseline. However, performance varied depending on the architecture, input representation, and evaluation domain.

7.2 Answers to the Research Questions

The project was organized around four research questions, each addressing a different aspect of LLM authorship classification.

RQ1: Can CNN and LSTM classifiers identify the generating LLM from its response?

Our experiments demonstrated that both architectures could distinguish between GPT, Claude, and Gemini responses with accuracy above the 33.33% chance baseline.

The CNN achieved 82.72% test accuracy and a macro-F1 score of 0.8293, while the LSTM achieved 71.60% accuracy and a macro-F1 score of 0.7141.

These results indicate that generated responses contained learnable patterns associated with their source models. The CNN achieved stronger performance under our experimental conditions, suggesting that local textual patterns were particularly useful for this classification task.

RQ2: Does including the original prompt improve classification performance?

Output-only input produced the strongest observed results for both architectures under the tested token allocation.

Input-only accuracy was exactly 33.33% for both models because identical prompts were paired with all three labels. The combined input/output configuration achieved 77.78% accuracy for the CNN and 53.09% for the LSTM.

Combined input performed worse under the tested token allocation. This comparison does not isolate the effect of adding the prompt because combined input also reduced the maximum response length from 512 to 383 tokens. A controlled comparison with equal response-token budgets, a shared training-derived vocabulary, and matching checkpoint-selection criteria is needed to assess the contribution of the prompt separately.

RQ3: Can the classifiers generalize to a previously unseen subject category?

The cross-domain experiment evaluated the models on 90 general-writing responses after excluding this category from training and validation.

The CNN achieved 72.22% accuracy and a macro-F1 score of 0.7161, while the LSTM achieved 56.67% accuracy and a macro-F1 score of 0.5429.

Both models performed above the 33.33% chance baseline on the unseen general-writing category. Within RQ3, CNN's accuracy decreased from 89.39% on source-domain responses to 72.22% on unseen general-writing responses, while the LSTM's accuracy increased from 51.52% to 56.67%. These results indicate that cross-domain behavior differed between the two architectures.

These results suggest that some learned patterns transferred across subject categories, but classification performance was affected by differences between the training and evaluation domains.

RQ4: Which writing characteristics may help distinguish the three LLM families?

Our RQ4 analysis examined measurable linguistic and structural characteristics across all 540 responses. Claude produced the longest responses on average (629.71 words), GPT averaged 459.27 words, and Gemini produced the shortest responses (328.78 words).

Claude used list markers most frequently (61.67%), while Gemini had the highest frequency of heading markers (39.44%), code fences (23.89%), and bold formatting markers (46.67%).

Lexical diversity, measured using MATTR-50, was highest for Gemini (0.8625), followed by GPT (0.8592) and Claude (0.8259). These results demonstrate measurable differences in response length, vocabulary diversity, and formatting. However, this descriptive analysis does not establish which features the trained classifiers relied upon when making predictions.

Overall, the four research questions demonstrated that LLM authorship classification is feasible within our experimental setting, while highlighting the importance of input selection, architecture choice, and generalization to unseen data.

7.3 Key Contributions and Lessons Learned

One of the primary contributions of this project was the construction of an original dataset specifically designed for comparing responses generated by GPT, Claude, and Gemini.

By using identical prompts across all three models and maintaining balanced class distributions, we established a consistent foundation for evaluating authorship classification.

A second contribution was the implementation and comparison of two different neural-network architectures. Developing both CNN and LSTM classifiers allowed us to investigate how different approaches to processing text performed on the same classification problem.

The project also compared original prompts, generated responses, and combined prompt–response text. This comparison highlighted the need to control response-token budgets, vocabulary construction, and checkpoint selection before attributing a performance change to prompt inclusion.

Another contribution was the evaluation of cross-domain generalization. By holding out the general-writing category, we examined whether models trained on other subjects could recognize the source of responses from a previously unseen category.

Beyond the experimental results, the project provided practical experience with the complete machine-learning development process. This included dataset construction, preprocessing, tokenization, vocabulary development, neural-network implementation, hyperparameter selection, model training, evaluation, and interpretation.

The collaboration between team members also strengthened our understanding of reproducible experimentation and the importance of maintaining consistent datasets and evaluation procedures when comparing machine-learning models.

One of the most important lessons was that strong performance on a held-out test set does not necessarily guarantee equally strong performance under a change in subject domain.

We also learned that model complexity alone does not determine classification effectiveness. Under our experimental conditions, the CNN achieved better performance than the LSTM, demonstrating the importance of evaluating architectural choices empirically.

7.4 Future Research Directions

Although our project demonstrated promising results, several directions could extend the investigation.

Future work could expand the dataset to include more prompts, additional subject categories, and a wider range of language models. This would help determine whether the observed classification patterns remain reliable across more diverse text-generation tasks.

Another direction would be to investigate transformer-based classification architectures and compare their performance with the CNN and LSTM models implemented in this project.

Further experiments could also examine whether classifiers can distinguish between different versions of the same language model family. This would be particularly relevant as LLM architectures and generation capabilities continue to evolve.

A more comprehensive cross-domain evaluation could be conducted by holding out each dataset category individually and measuring performance across multiple unseen domains.

Future research could also investigate the effects of response length, text formatting, generation settings, and preprocessing choices on classification accuracy.

Finally, combining neural-network predictions with interpretable linguistic features could help explain which characteristics contribute most strongly to authorship classification.

7.5 Final Conclusion

In conclusion, this project demonstrated that neural-network models can learn identifiable patterns from text generated by different large language model families.

Using an original dataset of 540 responses, we successfully implemented and evaluated CNN and LSTM classifiers for distinguishing between GPT, Claude, and Gemini.

The CNN achieved the strongest overall performance in our experiments, reaching 82.72% accuracy on the primary classification task and 72.22% accuracy on the held-out general-writing category.

Generated responses supported classification above the balanced chance baseline, whereas identical prompts paired with all three labels forced input-only accuracy to one-third. The combined-input results do not isolate the effect of adding prompts because they also reduced the response-token budget from 512 to 383. Cross-domain behavior differed: CNN accuracy decreased on general writing, while LSTM accuracy increased relative to its weaker source-domain result. The reported performance gaps are descriptive and were not established as statistically significant.

While the study was limited by dataset size, model coverage, and evaluation scope, it provided evidence that machine-learning techniques can identify source-related patterns in LLM-generated text.

More broadly, the project highlighted both the potential of automated LLM authorship classification and the need for careful evaluation before applying such systems to unfamiliar models, subjects, or generation conditions.

References

[1] Y. Kim, “Convolutional Neural Networks for Sentence Classification,” in *Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2014, pp. 1746–1751. https://doi.org/10.3115/v1/D14-1181

[2] S. Hochreiter and J. Schmidhuber, “Long Short-Term Memory,” *Neural Computation*, vol. 9, no. 8, pp. 1735–1780, 1997. https://doi.org/10.1162/neco.1997.9.8.1735

[3] D. P. Kingma and J. Ba, “Adam: A Method for Stochastic Optimization,” in *Proceedings of the 3rd International Conference on Learning Representations (ICLR)*, 2015. https://arxiv.org/abs/1412.6980

[4] A. Paszke et al., “PyTorch: An Imperative Style, High-Performance Deep Learning Library,” in *Advances in Neural Information Processing Systems*, vol. 32, 2019. https://arxiv.org/abs/1912.01703

[5] M. A. Covington and J. D. McFall, “Cutting the Gordian Knot: The Moving-Average Type–Token Ratio (MATTR),” Journal of Quantitative Linguistics, vol. 17, no. 2, pp. 94–100, 2010. https://doi.org/10.1080/09296171003643098